



**Pontificia Universidad
Católica del Ecuador**
Seréis mis testigos

MANABÍ

**PONTIFICIA UNIVERSIDAD CATÓLICA DEL ECUADOR
SEDE MANABÍ**

CARRERA DE SOFTWARE

TRABAJO DE TITULACIÓN

**DESARROLLO DE UNA PLATAFORMA SAAS QUE INTEGRA
MÓDULOS CON IA PARA LA GESTIÓN INTEGRAL DE PYMES
GASTRONÓMICAS EN PORTOVIEJO**

LÍNEA DE INVESTIGACIÓN

TECNOLOGÍA DE LA INFORMACIÓN Y LA COMUNICACIÓN

SUBLÍNEA DE INVESTIGACIÓN

INGENIERÍA DE SOFTWARE, INNOVACIÓN Y EMPRENDIMIENTO EN TIC

**PREVIO AL TÍTULO DE
INGENIERO EN SOFTWARE**

AUTORES

**LUIGGI ANDREE ARTEAGA SANTANA
CHRISTIAN ORLANDO SARAGURO ENCALADA**

TUTOR

JOSE NARANJO, M.ENG.

PORTOVIEJO, FEBRERO 2026

Certificación del Tutor

En mi calidad de tutor del trabajo de integración curricular, certifico haber revisado el presente manuscrito de investigación, el mismo que se ajusta a las normas vigentes de la Pontificia Universidad Católica del Ecuador Sede Manabí, cumpliendo la Normativa del Trabajo de Integración Curricular; en consecuencia, es apto para su presentación y sustentación. En la ciudad de Portoviejo a los quince días del mes de febrero de dos mil veintiséis.

JOSE NARANJO, M.ENG.

CI: 1003528674

Tutor

Acta de aprobación del Tribunal

El jurado examinador aprueba el presente trabajo de integración curricular en nombre de la Pontificia Universidad Católica del Ecuador, Sede Manabí.

En la ciudad de Portoviejo a los quince días del mes de febrero de dos mil veintiséis.

Nombre del revisor 1

CI:0000000000

Lector Revisor1(a)

Nombre del revisor 2

CI:0000000000

Lector Revisor2(a)

JOSE NARANJO, M.ENG

CI: 1003528674

Tutor

Declaración de Originalidad

Este manuscrito no contiene ningún tipo de material que ha sido aceptado para la obtención de un título universitario en otra institución, excepto en forma de información de soporte que ha sido debidamente citada en mi trabajo. Este trabajo es de total responsabilidad del autor, quien declara bajo juramento que ninguna sección de este trabajo de integración curricular infringe los derechos de autor de nadie.

En la ciudad de Portoviejo a los quince días del mes de febrero de dos mil veintiséis.

LUIGGI ARTEAGA

C.I.: 1314693597

Autor

CHRISTIAN SARAGURO

C.I.: 1350274484

Autor

Declaración de Derecho del Autor

Autorizo a la Pontificia Universidad Católica del Ecuador a distribuir este manuscrito de investigación en medios físicos y electrónicos con el fin de promover la divulgación de mis resultados a la comunidad científica y a la sociedad en general. Adicionalmente autorizo el uso de los contenidos de esta investigación como bibliografía para fines académicos, por cualquier medio o procedimiento, citando como fuente de información al autor de este trabajo. En la ciudad de Portoviejo a los quince días del mes de febrero de dos mil veintiséis.

LUIGGI ARTEAGA

C.I.: 1314693597

Autor

CHRISTIAN SARAGURO

C.I.: 1350274484

Autor

Dedicatoria

Dedico este trabajo, en primer lugar, a Dios, por ser mi fortaleza en los momentos difíciles y por permitirme alcanzar una de las metas más importantes de mi vida.

A mi familia, por su amor incondicional, sacrificio, paciencia y por creer siempre en mí, incluso cuando yo mismo dudaba. Su apoyo constante ha sido el motor que me impulsó a seguir adelante.

A mis padres, por ser ejemplo de esfuerzo, responsabilidad y perseverancia, y por enseñarme que los sueños se alcanzan con trabajo y dedicación.

Finalmente, dedico este logro a todas aquellas personas que con una palabra de aliento, un consejo o un gesto de apoyo hicieron posible la culminación de esta etapa tan significativa en mi vida.

Agradecimientos

Agradezco profundamente a Dios por brindarme la fortaleza, salud y sabiduría necesarias para culminar esta importante etapa de mi formación profesional.

Mi sincero agradecimiento a mi familia, quienes han sido mi mayor apoyo y motivación constante, brindándome su amor, comprensión y respaldo incondicional en cada paso de este proceso.

Agradezco de manera especial a mis docentes, quienes con su conocimiento, paciencia y orientación contribuyeron significativamente a mi desarrollo académico y profesional, permitiéndome adquirir las herramientas necesarias para la realización de este trabajo.

Un agradecimiento particular a mi tutor/a de tesis, por su guía, tiempo y valiosos aportes durante el desarrollo de esta investigación, siendo un pilar fundamental para su culminación exitosa.

Finalmente, agradezco a todas las personas que, de manera directa o indirecta, colaboraron en la realización de este proyecto y formaron parte de este importante logro personal y profesional.

Resumen

Este estudio aborda la limitada digitalización de los procesos operativos en las PYMES gastronómicas de Portoviejo y presenta el diseño, desarrollo y validación técnica de una plataforma SaaS modular para su gestión. La solución integra backend (NestJS), panel web y app móvil con módulos de pedidos, inventarios, repartidores, vista de cocina y dashboard analítico, además de un chatbot RAG por WhatsApp. Los medios de pago contemplados son efectivo y transferencia. Los requisitos se levantaron mediante literatura y entrevistas; el cumplimiento se verificó con pruebas de integración y escenarios de prueba en *staging*. La usabilidad se evaluó con la escala SUS en una PYME piloto. Los resultados muestran viabilidad técnica y usabilidad aceptable en entorno controlado: puntaje SUS de 70 (umbral de aceptabilidad ≥ 68), tasa de éxito en flujos críticos del 96 %, tiempo promedio de registro de pedidos de 3 min y tasa de errores del 9 %, cumpliendo los umbrales definidos. Se aporta así una alternativa de digitalización alineada con la Agenda de Transformación Digital 2022–2025 (Ministerio de Telecomunicaciones y de la Sociedad de la Información, 2022).

Palabras clave: PYMES, gastronomía, digitalización, SaaS, Portoviejo.

Abstract

This study addresses the limited digitalization of operational processes in gastronomic SMEs in Portoviejo and presents the design, development, and technical validation of a modular SaaS platform for their management. The solution integrates a NestJS backend, web panel, and mobile app with order management, inventory, delivery, kitchen view, and analytics, plus a WhatsApp RAG chatbot. Payment methods are cash and transfer only. Requirements were collected through literature review and interviews; compliance was verified via integration tests and staged scenarios. Usability was assessed with the SUS scale in one pilot SME. Results show technical feasibility and acceptable usability in a controlled environment: SUS score of 70 (acceptability threshold ≥ 68), 96 % success rate in critical flows, 3 min average order registration time, and 9 % error rate, meeting the defined thresholds. The work provides a digitalization alternative aligned with Ecuador's 2022–2025 Digital Transformation Agenda.

Keywords: SMEs, gastronomy, digitalization, SaaS, Portoviejo.

Tabla de Contenido

Índice de apéndices	IX
Introducción	1
Antecedentes	2
Contexto del sector gastronómico en Portoviejo	2
Composición empresarial y digitalización en Ecuador	2
Barreras para la digitalización de PYMES	3
Políticas nacionales de transformación digital	3
Plataformas SaaS y digitalización en el sector gastronómico	4
Justificación de una solución SaaS para PYMES gastronómicas	4
Definición del problema	5
Preguntas de investigación	7
Pregunta principal	7
Preguntas específicas	7
Objetivos	8
Objetivo general	8
Objetivos específicos	8
Alcance	9

Flujo general de la plataforma	10
Método	12
Diseño de la Investigación	12
Enfoque y tipo	12
Población y Muestra	13
Población	13
Muestra y criterios de selección	13
Operacionalización de variables e indicadores	13
Metodología	16
Requerimientos funcionales y no funcionales (Objetivo específico 1)	18
Técnicas e instrumentos de recolección de datos	21
Consideraciones éticas	23
Procedimiento	24
ETAPA 1: Planificación	24
Arquitectura de la Plataforma	27
Arquitectura de la aplicación móvil	28
Arquitectura de la aplicación web	28
Arquitectura del backend	28
Arquitectura de la base de datos	31
Roles, permisos y seguridad (JWT, RBAC)	35
ETAPA 2: Desarrollo	37

Desarrollo de la Plataforma SaaS	37
Desarrollo del frontend móvil (React Native + Expo)	37
Desarrollo del frontend panel web (Next.js + Tailwind CSS)	39
Desarrollo del backend (NestJS, capas, módulos)	41
ETAPA 3: Validación y Despliegue	47
Arquitectura del despliegue	47
Seguridad operativa y configuración de despliegue	48
Despliegue en <i>staging</i> y validación técnica	49
Resultados	50
Introducción	50
Resultados del diagnóstico de digitalización	51
Caracterización de la PYME piloto	51
Estado de digitalización previo	51
Resultados de la validación de usabilidad y adopción	53
Usabilidad del sistema	53
Desempeño del sistema en escenarios de prueba	56
Resultados técnicos del software	57
Pruebas unitarias y de integración	57
Métricas de disponibilidad y rendimiento	57
Validación de funcionalidades	59
Problemas identificados y soluciones	60

Discusión	62
Objetivo General: La plataforma SaaS integrada y el significado de los hallazgos	63
Análisis de Objetivos Específicos y Contraste Bibliográfico	64
Identificación y documentación de requisitos (Objetivo Específico 1)	64
Diseño de arquitectura técnica de la plataforma (Objetivo Específico 2)	64
Desarrollo de módulos funcionales principales (Objetivo Específico 3)	65
Implementación e integración del chatbot RAG por WhatsApp (Objetivo Específico 4)	65
Verificación del cumplimiento de requisitos funcionales (Objetivo Específico 5) . .	66
Evaluación de usabilidad con escala SUS (Objetivo Específico 6)	66
Limitaciones de la Investigación	67
Conclusiones	69
Referencias bibliográficas	75
Apéndice A	76
Apéndice B	78
Apéndice C	79
Apéndice D	81
Apéndice E	82
Apéndice F	83

Apéndice G	85
Apéndice H	87
Apéndice I	88
Apéndice J	90
Apéndice K	93
Apéndice L	94

Lista de Figuras

1.	Diagrama de flujo del funcionamiento general de la plataforma SaaS. Fuente: elaboración propia.	11
2.	Ciclo iterativo de la Investigación Basada en Diseño (Fases 1–6). Fuente: elaboración propia con base en Hevner et al., 2004; Peffers et al., 2007.	17
3.	Arquitectura por capas de la plataforma SaaS	27
4.	Arquitectura de la aplicación móvil	29
5.	Arquitectura de la aplicación web	30
6.	Arquitectura del backend (API REST)	30
7.	Diagrama Entidad-Relación (ER) de la base de datos	35
8.	Flujo de autenticación y autorización (JWT + RBAC)	37
9.	Flujo de la interfaz móvil por rol (cliente, repartidor, cocina)	39
10.	Flujo del panel web administrativo	41
11.	Diagrama de secuencia: flujo completo del proceso de pedido. Fuente: elaboración propia.	45
12.	Lógica del módulo de inventarios y control de stock	45
13.	Flujo del módulo de repartidores	45
14.	Arquitectura de despliegue (VPS, Vercel, APK)	48
15.	Diagrama entidad–relación de la base de datos de la plataforma SaaS	76

16.	Arquitectura por capas de la plataforma SaaS	78
17.	Arquitectura de la aplicación móvil	79
18.	Arquitectura del panel web	80
19.	Arquitectura modular del backend (NestJS) planificada	81
20.	Flujo de autenticación y autorización (JWT + RBAC)	82
21.	Flujo principal del frontend móvil para cliente, repartidor y cocina	83
22.	Flujo principal del panel web administrativo	84
23.	Flujo general del proceso de pedido	85
24.	Diagrama del módulo de inventario	86
25.	Diagrama del módulo de repartidores	86
26.	Arquitectura del despliegue de la plataforma	87
27.	Diagrama de red y arquitectura de la plataforma SaaS web y móvil	88

Lista de Tablas

1.	Operacionalización de variables e indicadores de medición	15
2.	Requerimientos funcionales y no funcionales priorizados	18
3.	Planificación de sprints del desarrollo	20
4.	Técnicas e instrumentos de recolección de datos	22
5.	Selección tecnológica para la arquitectura de la plataforma SaaS	25
6.	Estado de digitalización previo: eventos y porcentajes por ítem	52
7.	Puntajes SUS individuales y promedio por rol ($n = 6$ usuarios)	53
8.	Desempeño en tareas guiadas por rol de usuario	55
9.	Resultados de métricas de desempeño en escenarios de prueba frente a umbrales de éxito	56
10.	Resumen de pruebas unitarias (muestra trazable)	58
11.	Validación de funcionalidades por casos de prueba y criterio de aceptación	59
12.	Tareas evaluadas en pruebas de usabilidad	91
13.	Evidencia resumida de pruebas unitarias y de integración	92
14.	Detalle de arquitectura por capas y componentes	94

Índice de apéndices

Apéndice	Descripción	Página
Apéndice A	Diagrama de la base de datos	76
Apéndice B	Arquitectura por capas de la plataforma	78
Apéndice C	Arquitectura de la aplicación móvil y del panel web	79
Apéndice D	Arquitectura modular del backend	81
Apéndice E	Flujo de autenticación y autorización (JWT + RBAC)	82
Apéndice F	Flujo principal del frontend móvil y del panel web	83
Apéndice G	Flujo general del proceso de pedido y módulos operativos	85
Apéndice H	Arquitectura del despliegue	87
Apéndice I	Diagrama de red del sistema	88
Apéndice J	Procedimiento detallado de validación en <i>staging</i>	90
Apéndice K	Guía de entrevista semiestructurada (diagnóstico)	93
Apéndice L	Arquitectura detallada de la plataforma	94

Introducción

En las últimas décadas, la digitalización empresarial ha transformado radicalmente los modelos operativos y competitivos a nivel mundial. La adopción masiva de tecnologías digitales ha permitido a las organizaciones optimizar procesos, reducir costos operativos, mejorar la experiencia del cliente y tomar decisiones basadas en datos en tiempo real. Sectores como el comercio electrónico, la logística, los servicios financieros y la manufactura han experimentado cambios estructurales profundos mediante la implementación de sistemas de planificación de recursos empresariales (ERP, por sus siglas en inglés *Enterprise Resource Planning*), plataformas de gestión integrada, automatización de procesos e inteligencia artificial aplicada. Sin embargo, esta transformación digital no ha sido uniforme ni equitativa: mientras que grandes corporaciones disponen de recursos técnicos, económicos y humanos para adoptar soluciones tecnológicas avanzadas, las Pequeñas y Medianas Empresas (PYMES) continúan enfrentando barreras significativas de acceso, costo, capacitación y adaptabilidad tecnológica.

A nivel latinoamericano y particularmente en Ecuador, las PYMES constituyen la columna vertebral del tejido empresarial. Según Instituto Nacional de Estadística y Censos (2023), las unidades productivas clasificadas como micro, pequeñas y medianas empresas representan más del 90 % del tejido empresarial registrado y generan una proporción significativa del empleo nacional. No obstante, diversos estudios e informes institucionales (Ayora Recalde et al., 2024; Durán Mero, 2024; Ministerio de Telecomunicaciones y de la Sociedad de la Información, 2022) evidencian que la mayoría de estas empresas mantiene niveles de digitalización limitados, especialmente en sectores tradicionales como la gastronomía, donde los procesos operativos continúan dependiendo de gestión manual, canales de comunicación no integrados y ausencia de herramientas automatizadas para la toma de decisiones. Esta brecha digital no solo limita su competitividad

frente a empresas tecnificadas o plataformas digitales consolidadas, sino que también afecta su capacidad de adaptación, escalabilidad y sostenibilidad en mercados cada vez más exigentes, donde los consumidores demandan inmediatez, conveniencia, transparencia y experiencias omnicanal.

Frente a este panorama, las soluciones SaaS (Software as a Service) representan una oportunidad estratégica para democratizar el acceso a tecnología empresarial avanzada sin requerir inversiones iniciales elevadas, infraestructura propia o conocimientos técnicos especializados. Bajo esta perspectiva, el presente estudio se centra en evaluar la factibilidad, diseño, desarrollo y validación de una plataforma SaaS integral orientada específicamente a las PYMES gastronómicas de Portoviejo, una ciudad reconocida internacionalmente por la UNESCO como Ciudad Creativa de la Gastronomía (UNESCO, 2019), pero cuyos negocios gastronómicos locales aún enfrentan desafíos significativos en su proceso de modernización digital.

Antecedentes

Contexto del sector gastronómico en Portoviejo

Portoviejo fue declarada Ciudad Creativa de la Gastronomía por la UNESCO en 2019 (UNESCO, 2019), un reconocimiento internacional que posiciona a la gastronomía local como un elemento estratégico tanto cultural como económico. Este distintivo subraya el potencial del sector para impulsar el desarrollo turístico, generar empleo y fortalecer la identidad regional. Las PYMES gastronómicas constituyen el núcleo de este sector, desempeñando un rol fundamental en la preservación de tradiciones culinarias y en la dinamización de la economía local.

Composición empresarial y digitalización en Ecuador

A nivel nacional, el tejido empresarial ecuatoriano está compuesto en gran parte por PYMES. Según datos del sondeo de adopción digital realizado por Otecel S.A., cerca del 91 % de las PYMES ecuatorianas planea invertir en digitalización (Movistar Empresas / Telefónica Ecuador,

2024). En el caso de la provincia de Manabí, donde se encuentra Portoviejo, las PYMES también juegan un rol importante en la generación de empleo y el desarrollo local.

Barreras para la digitalización de PYMES

Uno de los desafíos principales para estas empresas es la limitada adopción de tecnologías digitales. Si bien existe un reconocimiento creciente de su importancia, muchas PYMES siguen enfrentando barreras estructurales como costos elevados, falta de conocimientos técnicos, ausencia de capacitación especializada y sistemas tecnológicos poco adaptados a sus necesidades (Ayora Recalde et al., 2024; Durán Mero, 2024).

En muchos negocios gastronómicos pequeños, la gestión operativa (por ejemplo, inventarios y pedidos) continúa siendo manual o con herramientas básicas. Esta falta de automatización puede generar errores, desperdicios y dificultades para proyectar la demanda, lo que impacta negativamente en la rentabilidad y eficiencia. Además, el servicio al cliente todavía depende de canales tradicionales como llamadas telefónicas o WhatsApp, limitando la capacidad de ofrecer una experiencia profesional, rápida y escalable.

Por otra parte, el beneficio de adoptar métodos de pago digital y sistemas integrados es significativo para incrementar eficiencia operativa (Vistazo, 2025). Aunque no existen estudios exactos para todas las PYMES gastronómicas de Portoviejo, análisis de mercado muestran que la adopción de soluciones digitales puede reducir costos operativos y mejorar márgenes de pequeños emprendimientos.

Políticas nacionales de transformación digital

La Agenda de Transformación Digital del Ecuador 2022–2025 (Ministerio de Telecomunicaciones y de la Sociedad de la Información, 2022) establece una hoja de ruta institucional para impulsar la adopción de tecnologías en distintos sectores, incluyendo las PYMES. Entre sus objetivos está promover infraestructura tecnológica, facilitar la capacitación y crear un entorno favorable para que

las empresas pequeñas accedan a soluciones digitales.

Plataformas SaaS y digitalización en el sector gastronómico

En el ámbito internacional, existen investigaciones previas sobre plataformas SaaS aplicadas al sector de la restauración y la gastronomía, que abordan la gestión de pedidos, reservas e inventarios en formato cloud (Durán Mero, 2024). Estos trabajos destacan la conveniencia del modelo SaaS para reducir costos de implantación y mantenimiento en negocios de menor escala. En el contexto ecuatoriano, los estudios sobre digitalización en PYMES gastronómicas son aún escasos; no obstante, informes como el de Otecel S.A. (Movistar Empresas / Telefónica Ecuador, 2024) y la Agenda de Transformación Digital (Ministerio de Telecomunicaciones y de la Sociedad de la Información, 2022) evidencian el interés creciente de las PYMES por adoptar herramientas digitales, incluyendo sistemas de gestión operativa. La literatura sobre evaluación de usabilidad en soluciones SaaS indica que escalas estandarizadas como el System Usability Scale (SUS) permiten cuantificar la facilidad de uso percibida y comparar artefactos en fase de validación (Ayora Recalde et al., 2024). Estos antecedentes sustentan la pertinencia de desarrollar y validar una plataforma SaaS orientada específicamente a las PYMES gastronómicas de Portoviejo.

Justificación de una solución SaaS para PYMES gastronómicas

Frente a estas problemáticas, una solución SaaS (Software como Servicio) adaptada a PYMES gastronómicas locales emerge como una alternativa viable y estratégica. Un sistema SaaS puede ofrecer módulos integrados para la gestión de pedidos, control de inventarios, analítica empresarial y atención al cliente automatizada (mediante chatbots), sin requerir inversiones grandes en infraestructura o conocimientos técnicos avanzados por parte de los usuarios.

Por lo tanto, desarrollar una plataforma SaaS que responda a las necesidades operativas, técnicas y económicas de las PYMES gastronómicas de Portoviejo constituye una estrategia viable y alineada con las metas institucionales nacionales de digitalización y con las aspiraciones locales de

modernización, competitividad y crecimiento sostenible del sector.

Definición del problema

En el sector gastronómico de la ciudad de Portoviejo, las Pequeñas y Medianas Empresas (PYMES) desempeñan un rol fundamental en la economía local, aportando significativamente a la generación de empleo, al dinamismo comercial y a la identidad gastronómica de la región. A pesar de su importancia, este sector enfrenta un proceso de digitalización lento e incompleto, caracterizado por la utilización de métodos tradicionales en la gestión operativa, administrativa y comercial. En un entorno donde la tecnología se ha convertido en un componente esencial para la competitividad y la sostenibilidad, muchas de estas empresas continúan utilizando procesos manuales para administrar pedidos, inventarios y servicios de delivery, dificultando su adaptación a las nuevas exigencias del mercado digital.

El problema principal identificado es la falta de soluciones tecnológicas accesibles, integradas y adaptadas al contexto local, que permitan a las PYMES gastronómicas gestionar de forma eficiente sus operaciones diarias. Actualmente, la mayoría de estos negocios no dispone de herramientas centralizadas para controlar inventarios, administrar pedidos, gestionar repartidores o brindar atención al cliente mediante sistemas automatizados. Esto genera desorganización, tiempos de respuesta elevados y errores en la toma de pedidos que afectan directamente la calidad del servicio. La ausencia de una plataforma tecnológica integral limita la capacidad de estas empresas para modernizarse y competir frente a cadenas más grandes o plataformas de delivery externas.

Diversos estudios y diagnósticos sobre transformación digital en Ecuador evidencian que las PYMES mantienen un nivel de adopción tecnológica limitado, especialmente en actividades tradicionales como la gastronomía. Aunque representan más del 90 % del tejido empresarial del país (Instituto Nacional de Estadística y Censos, 2023), informes como la Agenda de Transformación Digital del Ecuador 2022–2025 (Ministerio de Telecomunicaciones y de la Sociedad de la Información, 2022) señalan que estas empresas aún enfrentan barreras significativas para

modernizar sus procesos, debido a limitaciones económicas, escasa capacitación técnica y la falta de soluciones accesibles y adaptadas a su escala operativa. En Portoviejo, pese a ser reconocida por la UNESCO como Ciudad Creativa de la Gastronomía en 2019 (UNESCO, 2019), la mayoría de los negocios gastronómicos continúa operando sin sistemas integrados de gestión para pedidos, inventarios y atención al cliente, lo cual genera procesos manuales, inconsistencias y baja eficiencia operativa. Esta evidencia muestra una brecha marcada entre el potencial gastronómico de la ciudad y el nivel real de digitalización presente en sus pequeñas y medianas empresas, lo que confirma la existencia de un problema estructural que afecta su competitividad y sostenibilidad.

Las causas del problema se relacionan principalmente con:

1. El alto costo de las soluciones tecnológicas disponibles en el mercado, que suelen estar orientadas a empresas medianas o grandes y no se ajustan al presupuesto de las PYMES locales.
2. La falta de capacitación tecnológica del personal, lo cual dificulta la adopción de plataformas complejas.
3. La ausencia de una oferta local de software adaptable, que incluya en una sola plataforma módulos de delivery, inventarios, atención automatizada y analítica.
4. La dependencia de plataformas de terceros, que cobran comisiones elevadas y no permiten gestionar los procesos de manera personalizada.

Estas causas se combinan para generar un rezago tecnológico que limita el crecimiento y la modernización del sector gastronómico.

Si el problema no se soluciona, las PYMES gastronómicas de Portoviejo continuarán enfrentando dificultades operativas que limitan la calidad del servicio y la experiencia del cliente. La falta de digitalización provocará que estos negocios sigan siendo menos competitivos frente a empresas que ya adoptaron herramientas tecnológicas avanzadas. Además, persistirán los errores en la toma de

pedidos, las dificultades en el control de inventarios y los tiempos de entrega elevados debido a una gestión manual de repartidores. A largo plazo, esta situación limitará su capacidad para adaptarse a un mercado que demanda rapidez, precisión y experiencia digital.

Abordar este problema es fundamental para demostrar la viabilidad técnica de una plataforma SaaS accesible e integrada para el sector gastronómico local. El desarrollo de una solución modular permitirá evaluar si la digitalización de procesos operativos (gestión de pedidos, inventarios, repartidores y atención automatizada) es técnicamente factible y usable en el contexto de las PYMES de Portoviejo. Además, esta investigación se alinea con los objetivos nacionales de digitalización y con la visión de Portoviejo como Ciudad Creativa de la Gastronomía. Los resultados de esta validación técnica sentarán las bases para futuras investigaciones orientadas a la implementación a mayor escala y la medición de impacto operativo y económico en el sector.

En este contexto, surge la necesidad de desarrollar y validar una solución tecnológica integrada que permita evaluar su viabilidad técnica y usabilidad en el entorno específico de las PYMES gastronómicas de Portoviejo.

Preguntas de investigación

Pregunta principal

¿Cuál es la viabilidad técnica y el nivel de usabilidad percibida de una plataforma SaaS modular que integre gestión de pedidos, inventarios, repartidores y atención con inteligencia artificial, al validarse en un entorno controlado en PYMES gastronómicas de Portoviejo?

Preguntas específicas

1. ¿Cuáles son los requisitos funcionales y no funcionales que debe satisfacer una plataforma SaaS orientada a la gestión operativa de PYMES gastronómicas en el contexto de Portoviejo?

2. ¿Qué arquitectura de software y stack tecnológico resultan adecuados para implementar una solución modular, escalable y mantenible que integre gestión de pedidos, inventarios, repartidores y dashboard analítico?
3. ¿Cómo integrar un módulo de inteligencia artificial basado en tecnología RAG para automatizar la atención al cliente mediante WhatsApp?
4. ¿En qué medida la plataforma desarrollada cumple con los requisitos funcionales especificados y los criterios de calidad de software establecidos, y con qué métricas e indicadores se evaluará dicho cumplimiento (p. ej. porcentaje de requisitos cumplidos, pruebas de aceptación, dimensiones de calidad ISO/IEC 25010)?
5. ¿Cuál es el nivel de usabilidad percibida de la plataforma según evaluación con usuarios de prueba en entorno controlado (escala SUS)?

Objetivos

Objetivo general

Desarrollar una plataforma SaaS que integre módulos de gestión operativa e inteligencia artificial, para optimizar los procesos de pedidos, inventarios, delivery y atención al cliente de las PYMES gastronómicas de Portoviejo.

Objetivos específicos

1. Identificar y documentar los requisitos funcionales y no funcionales mediante revisión de literatura, análisis del dominio y entrevistas con actores del sector gastronómico.
2. Diseñar la arquitectura técnica de la plataforma, definiendo la estructura modular, patrones de diseño, modelo de datos y protocolos de seguridad e integración.

3. Desarrollar los módulos funcionales principales: gestión de pedidos con geolocalización, control de inventarios, administración de repartidores, vista de cocina y dashboard analítico con KPIs.
4. Implementar e integrar un chatbot basado en el modelo RAG accesible vía WhatsApp para automatizar respuestas a consultas frecuentes.
5. Verificar el cumplimiento de requisitos funcionales mediante pruebas de integración y validación de flujos críticos en entorno controlado (*staging*), con métricas de éxito definidas (p. ej. tasa de completitud $\geq 95\%$, tiempo de registro ≤ 3 min).
6. Evaluar la usabilidad de la plataforma mediante la aplicación de la escala SUS con usuarios de prueba en entorno controlado, con umbral de aceptabilidad ≥ 68 .

Alcance

El alcance del presente estudio se delimita en función de los entregables implementados y de las exclusiones del producto de investigación.

El trabajo comprende el desarrollo de una plataforma SaaS modular compuesta por backend (NestJS), aplicación móvil (React Native), panel web (Next.js) y base de datos PostgreSQL. La solución incluye seis módulos funcionales: gestión de pedidos con geolocalización, inventarios, repartidores, vista de cocina móvil, dashboard analítico con KPIs y un chatbot RAG por WhatsApp; los métodos de pago se limitan a efectivo y transferencia bancaria. El despliegue y la validación se realizaron en un entorno controlado (*staging*), con evidencia de pruebas de integración, ejecución de flujos críticos y evaluación de usabilidad mediante la escala SUS en una PYME piloto.

Quedan fuera del alcance la arquitectura multi-tenant (en este piloto se utiliza una instancia por cliente), la integración de una pasarela de pagos y la optimización automática de rutas, así como módulos adicionales (marketing, reseñas o proveedores), aplicación nativa iOS e infraestructura de alta disponibilidad. Asimismo, no se considera soporte post-implementación, un plan de

escalamiento operacional ni la medición del impacto económico o una evaluación a escala sectorial; el propósito es aportar evidencia de viabilidad técnica, funcional y de usabilidad en un entorno controlado, como base para futuras investigaciones a mayor escala.

Flujo general de la plataforma

Para contextualizar los entregables anteriores, en la Figura 1 se presenta el flujo de funcionamiento general de la plataforma SaaS. El diagrama muestra la secuencia de interacción entre el cliente (web o móvil), el panel de administración, la vista de cocina, el módulo de repartidores y el backend con base de datos, así como el canal de atención mediante chatbot por WhatsApp. Incluye los flujos de pedido (creación, preparación, asignación a repartidor y entrega) y la generación de datos para el dashboard analítico. La inclusión de esta figura responde a la necesidad de ofrecer una visión de conjunto del sistema antes de detallar cada componente en el capítulo Método.

Con base en esta vista general, el capítulo Método desarrolla posteriormente cada componente de arquitectura y su validación técnica de forma desagregada.

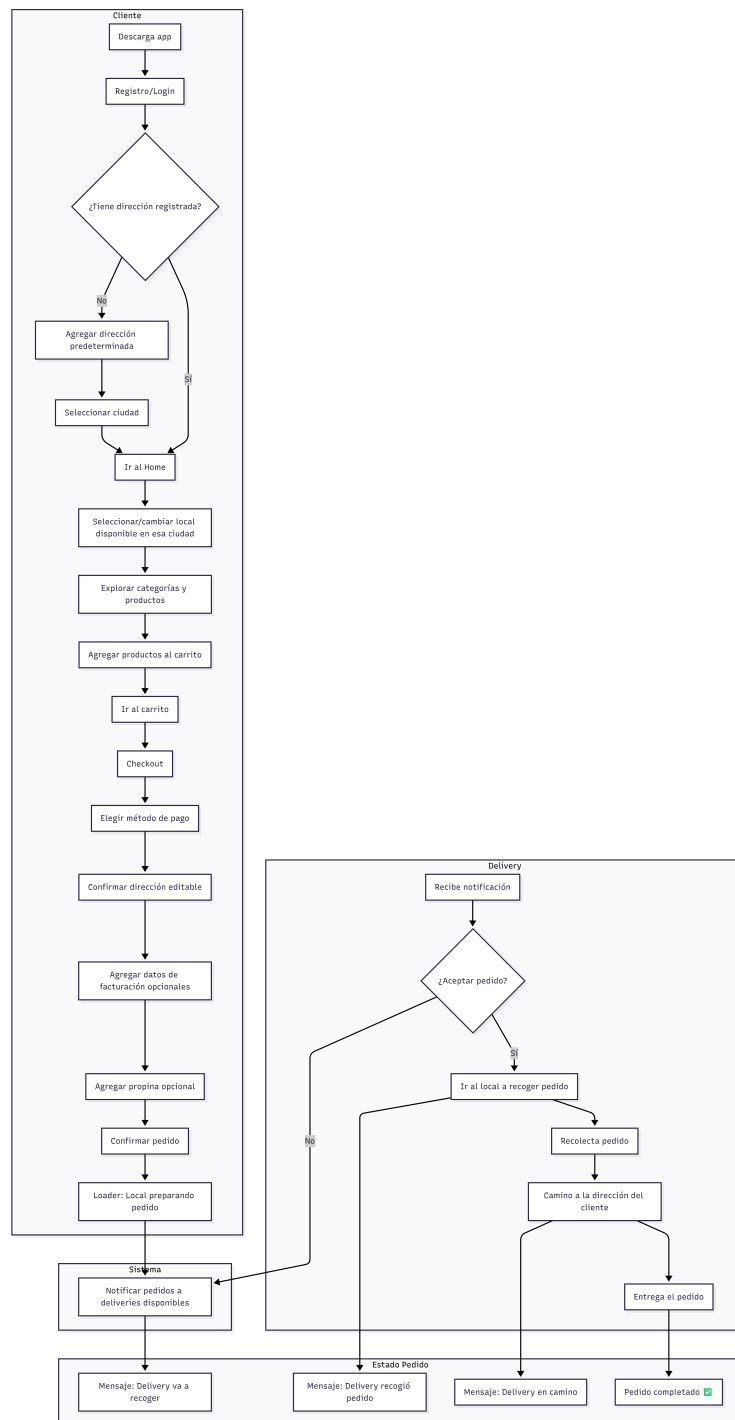


Figura 1

Diagrama de flujo del funcionamiento general de la plataforma SaaS. Fuente: elaboración propia.

Método

Diseño de la Investigación

Enfoque y tipo

La investigación se desarrolló bajo un enfoque de Investigación Basada en Diseño (Design Science Research), con evaluación de la solución mediante pruebas técnicas y pruebas de usabilidad en un entorno controlado (*staging*). Se adoptó un estudio de caso como contexto (PYME piloto), sin medición longitudinal de desempeño operativo real. Este enfoque se eligió para desarrollar y evaluar un artefacto de software (la plataforma SaaS) que resuelve una problemática concreta en un contexto específico.

El componente cualitativo permitió profundizar en las percepciones, necesidades y experiencias de los usuarios durante las pruebas de usabilidad, mientras que el cuantitativo posibilitó la medición objetiva de variables de desempeño en escenarios de prueba controlados. El estudio se clasifica como investigación aplicada de tipo tecnológica, dado que su propósito central es el diseño, construcción y validación técnica de un artefacto de software para demostrar su viabilidad en un contexto específico. El diseño es no experimental y transversal, ya que se observaron y describieron las variables en su contexto de prueba sin manipulación deliberada, en un período específico.

Población y Muestra

Población

La población objetivo estuvo constituida por la totalidad de las Pequeñas y Medianas Empresas (PYMES) del sector gastronómico formalmente establecidas en la ciudad de Portoviejo, provincia de Manabí, Ecuador.

Muestra y criterios de selección

Se aplicó un muestreo no probabilístico por conveniencia y criterio. La muestra final estuvo conformada por una PYME gastronómica. Los criterios de inclusión fueron:

- 1) Ser una PYME ubicada en Portoviejo.
- 2) Contar con servicio de consumo en el local y delivery.
- 3) Manifestar interés en mejorar procesos mediante digitalización.
- 4) Disponer de conectividad básica a internet.
- 5) Otorgar consentimiento para participar en todas las fases del estudio.

Se excluyeron establecimientos con operación irregular o que ya utilizaban sistemas de gestión integral.

La unidad de análisis se limita a un caso piloto; por tanto, los resultados no pretenden generalización estadística sino evidencia de viabilidad del artefacto.

Operacionalización de variables e indicadores

Para poder evaluar si la plataforma cumple los criterios de viabilidad técnica y usabilidad definidos en los objetivos, se operacionalizaron las variables dependientes en indicadores medibles con

instrumentos y umbrales de éxito. Esta operacionalización permite trazar los resultados del capítulo de Resultados con las preguntas de investigación y los objetivos. En la Tabla 1 se definen las variables, su definición operacional, el instrumento de medición utilizado y el umbral que se consideró como éxito en cada caso. Los valores de la tabla se utilizaron tanto para el diseño de las pruebas (Fase 5 de la metodología IBD) como para la interpretación de los resultados obtenidos.

Tabla 1*Operacionalización de variables e indicadores de medición*

Variable dependiente	Definición operacional	Instrumento de medición	Umbral de éxito
Eficiencia en el registro de pedidos	Tiempo y precisión en la captura y procesamiento de un pedido en tareas guiadas.	Ficha de observación estructurada (registro de tiempos y errores en tareas de prueba).	Tiempo promedio ≤ 3 min por pedido y tasa de errores $\leq 10\%$ en escenarios de prueba.
Eficacia del servicio de delivery	Capacidad del sistema para gestionar y coordinar entregas dentro de un tiempo esperado en escenarios de prueba.	Ficha de observación estructurada (registro de tiempos en flujos de prueba).	Tiempo promedio de entrega ≤ 35 min en escenarios de prueba.
Complejidad de flujos críticos	Capacidad del sistema para ejecutar los flujos principales (pedido, cocina, reparto) sin fallos en tareas guiadas.	Registros de pruebas de integración y observación de flujos (conteo de tareas completadas vs. fallidas).	Tasa de éxito $\geq 95\%$ en la finalización de tareas guiadas.
Estabilidad del sistema	Confiabilidad y continuidad del sistema durante las pruebas, minimizando fallos técnicos.	Registros de logs del sistema y observación directa durante pruebas.	Tasa de fallos técnicos $\leq 5\%$ durante el período de pruebas.

Variable dependiente	Definición operacional	Instrumento de medición	Umbral de éxito
Usabilidad del sistema	Grado de facilidad, eficiencia y satisfacción con que los usuarios pueden emplear la plataforma.	Cuestionario estandarizado System Usability Scale (SUS).	Puntaje promedio ≥ 68 (umbral de aceptabilidad en la escala SUS).

Nota: Los umbrales de éxito para las métricas de desempeño se establecieron con base en estándares razonables de calidad para un sistema en fase de validación técnica, considerando la factibilidad de ejecución en escenarios de prueba controlados. El umbral para el SUS se basa en la clasificación estándar de Bangor et al. (2009).

La Tabla 1 define explícitamente los criterios de éxito que luego se emplean en el capítulo de Resultados para verificar el cumplimiento de objetivos.

Metodología

El presente trabajo sigue la metodología Investigación Basada en Diseño (IBD), elegida por su capacidad para desarrollar y evaluar artefactos (productos, procesos, métodos o modelos) orientados a resolver problemas del mundo real (Hevner et al., 2004; Peffers et al., 2007). No existe un único modelo canónico de fases, ya que distintos autores han propuesto esquemas similares con variaciones; sin embargo, todas comparten un núcleo cíclico e iterativo de construcción y evaluación. Se presenta el siguiente esquema integrando los modelos clave de Hevner et al. (2004) y Peffers et al. (2007).

La Figura 2 comunica la lógica de avance del estudio: diagnóstico del problema (Fases 1 y 2), construcción técnica del artefacto (Fase 3), demostración y prueba en entorno controlado (Fase 4), evaluación formal frente a criterios (Fase 5) y comunicación de resultados (Fase 6). Su propósito es

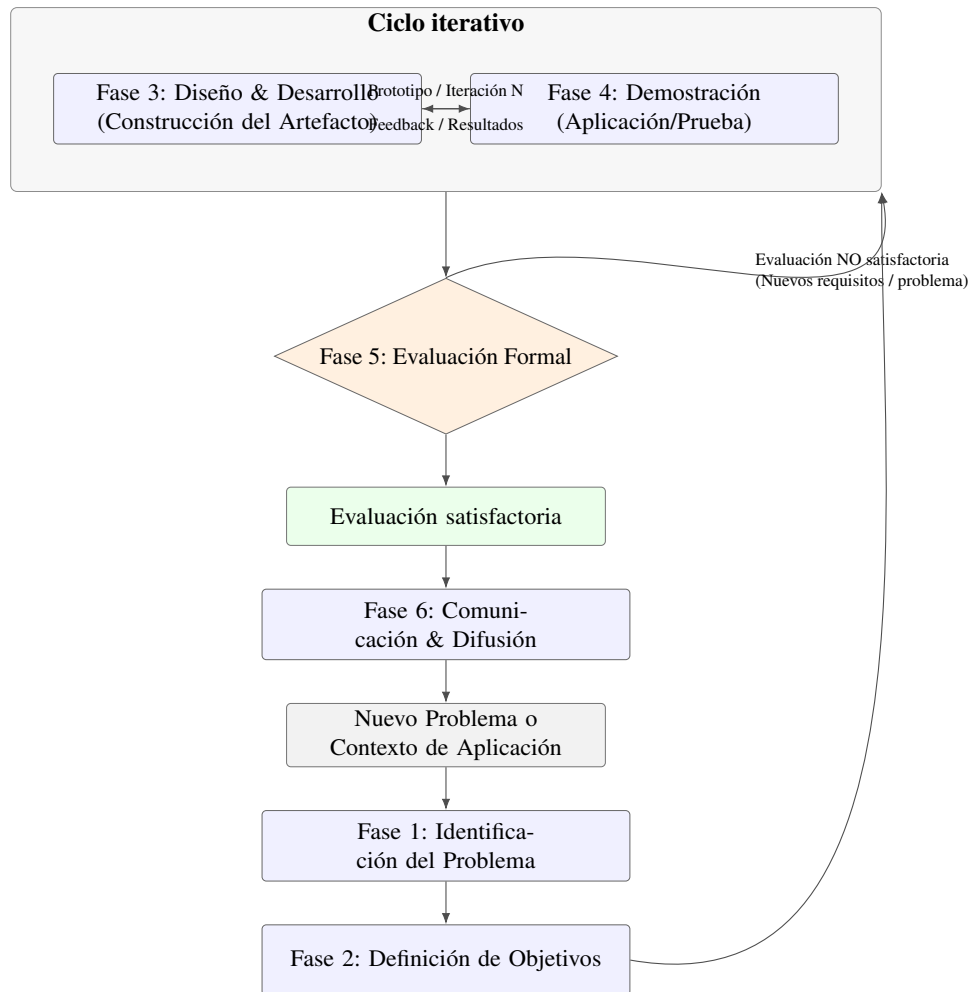


Figura 2

Ciclo iterativo de la Investigación Basada en Diseño (Fases 1–6). Fuente: elaboración propia con base en Hevner et al., 2004; Peffers et al., 2007.

mostrar el alcance metodológico completo y los elementos que se retroalimentan en cada iteración.

Fase 1. El objetivo de esta fase fue comprender y definir claramente el problema. Se realizó una revisión de la literatura del tema, contextualizándolo en el ámbito global y local.

Fase 2. Una vez comprendido el problema, se establecieron los objetivos que busca conseguir el producto. En esta fase se definieron los requisitos funcionales y no funcionales de la plataforma.

Requerimientos funcionales y no funcionales (Objetivo específico 1)

Con base en la entrevista semiestructurada, revisión documental y análisis del dominio, se levantaron los requerimientos que guiaron el diseño y la validación del prototipo. La Tabla 2 presenta una síntesis de los requerimientos funcionales (RF) y no funcionales (RNF) utilizados como referencia en las pruebas de integración y validación funcional.

Tabla 2

Requerimientos funcionales y no funcionales priorizados

Código	Descripción	Criterio de verificación
RF-ORD-01	Registrar pedidos con productos, cantidades y dirección de entrega.	El pedido se crea sin errores y queda visible para cocina/administración.
RF-ORD-02	Gestionar estados de pedido (pendiente, en preparación, listo, en camino, entregado).	El cambio de estado mantiene trazabilidad y secuencia válida.
RF-INV-01	Descontar inventario por pedido confirmado y evitar stock negativo.	Las existencias se actualizan de forma consistente tras cada operación.
RF-DEL-01	Asignar y gestionar pedidos para repartidores.	El repartidor visualiza pedidos y puede confirmar hitos de entrega.

Código	Descripción	Criterio de verificación
RF-ADM-01	Administrar catálogo (productos/categorías) desde panel web.	Operaciones CRUD ejecutan con validación y persistencia correctas.
RF-IA-01	Responder consultas frecuentes mediante chatbot por WhatsApp con contexto del negocio.	Respuestas coherentes para consultas de menú, horarios y estado.
RNF-SEG-01	Proteger endpoints con autenticación JWT y autorización por roles (RBAC).	Accesos no autorizados son bloqueados y registrados.
RNF-REN-01	Mantener tiempos de respuesta aceptables en entorno de prueba.	API dentro de umbrales definidos durante validación técnica.
RNF-DIS-01	Garantizar disponibilidad del servicio en <i>staging</i> .	Servicio operativo durante el período de pruebas con baja tasa de fallos.
RNF-USA-01	Alcanzar usabilidad aceptable medida con SUS.	Puntaje SUS promedio ≥ 68 en usuarios de prueba.

Fase 3 (Diseño y desarrollo del artefacto). En esta fase se realizan las actividades de construcción: selección del stack tecnológico, diseño de la arquitectura, desarrollo de cada módulo (backend, frontend web, aplicación móvil, integración con base de datos y con el chatbot). Cada incremento (prototipo, versión o módulo) se entrega como insumo para la siguiente fase.

Fase 4 (Demostración y prueba). En esta fase se demuestra y prueba cada incremento recibido de la Fase 3: despliegue en entorno piloto o *staging*, ejecución de escenarios de prueba con usuarios o en simulación, y registro de resultados (tiempos, errores, completitud de flujos). Los resultados de la demostración alimentan directamente el rediseño y el desarrollo de la siguiente iteración (retorno a Fase 3), cerrando el ciclo construcción-evaluación hasta alcanzar los criterios de aceptación definidos en la Fase 2.

En este estudio, las Fases 3 y 4 no se fusionan en una sola fase; se mantienen diferenciadas y se ejecutan de manera acoplada e iterativa. Primero se construye el incremento (Fase 3) y luego se demuestra/evalúa ese mismo incremento (Fase 4) antes de pasar al siguiente ciclo.

Proceso ágil y sprints

La implementación ágil se estructuró en iteraciones cortas (sprints) con objetivos definidos: levantar requisitos mínimos viables, construir prototipos funcionales, realizar pruebas de usabilidad y ajustar funcionalidades a partir de la retroalimentación. Cada sprint concluyó con una revisión donde se consolidaron mejoras y se definieron tareas para el siguiente ciclo.

Para evidenciar la planificación, la Tabla 3 resume la organización de sprints utilizada durante el desarrollo del prototipo.

Tabla 3
Planificación de sprints del desarrollo

Sprint	Duración	Objetivo principal	Entregables clave
Sprint 1	Semanas 1–2	Levantar requisitos y definir arquitectura base.	Backlog inicial, matriz RF/RNF, diseño de arquitectura.
Sprint 2	Semanas 3–4	Implementar núcleo backend y modelo de datos.	Módulos de autenticación, usuarios, productos y pedidos.
Sprint 3	Semanas 5–6	Construir interfaces móvil y web para flujos críticos.	Flujo de pedido cliente-cocina-repartidor, panel administrativo inicial.

Sprint	Duración	Objetivo principal	Entregables clave
Sprint 4	Semanas 7–8	Integrar módulos y ejecutar validación técnica/usabilidad.	Despliegue en <i>staging</i> , pruebas guiadas, ajustes finales.

Fase 5. Comparar los resultados de la demostración con los objetivos definidos en la Fase 2. Se utilizaron métricas cuantitativas y cualitativas, las mismas que se detallan en la Tabla 4 del presente capítulo.

Fase 6. En esta fase se realiza la comunicación de los resultados, los mismos que se definen en la sección Resultados del presente Trabajo.

Técnicas e instrumentos de recolección de datos

Las técnicas e instrumentos utilizados en cada fase del estudio se resumen en la Tabla 4. La tabla permite ubicar cada instrumento (entrevista semiestructurada, SUS, ficha de observación, checklist y logs) en el momento de aplicación y el público objetivo, lo que justifica su uso en el diagnóstico (Fase 1), la validación (Fases 4–5) y la comunicación de resultados (Fase 6). Los puntos clave son: la entrevista para el levantamiento de requisitos con dueños de PYMES; el SUS aplicado tras las tareas guiadas a usuarios de prueba; la ficha de observación para tiempos y errores durante las tareas; y la revisión documental para verificar despliegue y analizar logs.

Tabla 4*Técnicas e instrumentos de recolección de datos*

Técnica	Instrumento	Objetivo	Características	Momento de aplicación	Público objetivo
Entrevista	Guía de entrevista semiestructurada	Profundizar en procesos operativos, necesidades y expectativas; obtener requisitos funcionales y no funcionales y objetivos (general y específicos) mediante entrevistas con dueños de las PYMES.	Preguntas abiertas por áreas (pedidos, inventario, delivery). Transcrita.	Fase 1: Diagnóstico (línea base).	Dueños de las PYMES.
Prueba de usabilidad	Cuestionario SUS (System Usability Scale)	Medir la usabilidad percibida (facilidad de uso, satisfacción) de la plataforma en entorno de pruebas.	10 ítems con escala Likert de 5 puntos. Versión estandarizada en español. Confiabilidad ampliamente documentada.	Fase 6: Después de completar tareas guiadas en entorno controlado.	Usuarios de prueba de la PYME piloto (administrador, personal de cocina, repartidores).

Técnica	Instrumento	Objetivo	Características	Momento de aplicación	Público objetivo
Observación estructurada	Ficha de observación de tareas guiadas	Registrar datos cuantitativos sobre desempeño en escenarios de prueba (tiempos, errores, completitud de flujos).	Diseñada para registrar: tiempos (min), conteo de errores en tareas predefinidas, tasa de éxito en flujos críticos.	Durante la ejecución de tareas guiadas en entorno controlado (<i>staging</i>).	Usuarios de prueba ejecutando escenarios predefinidos.
Revisión documental	Checklist de despliegue y logs del sistema	Verificar el correcto despliegue técnico en <i>staging</i> y analizar logs de pruebas.	Lista de verificación para servicios en la nube. Análisis de logs y métricas técnicas durante las pruebas.	Durante y después de las pruebas en entorno controlado.	Plataforma SaaS desplegada en <i>staging</i> y sus registros de prueba.

Consideraciones éticas

Se respetaron principios éticos fundamentales en todo el estudio. La participación de las PYMES fue voluntaria; se garantizó la confidencialidad de la información y el uso de los datos solo con fines académicos. Se aseguró el anonimato de los participantes y se evitó la divulgación de información sensible.

Se aplicó consentimiento informado: se explicaron los objetivos del estudio, la naturaleza de las actividades y los derechos de los participantes (retirarse en cualquier momento, solicitar correcciones o eliminación de datos). La recolección y el almacenamiento de información siguieron

prácticas de minimización de datos, con acceso restringido y protección mediante credenciales. Los resultados se reportaron de forma agregada y sin identificadores, preservando la identidad de las PYMES. Se obtuvo la autorización de los responsables de cada negocio para la aplicación de instrumentos y la evaluación del prototipo. Se evaluó el riesgo de interferencia con las operaciones cotidianas y se planificaron las actividades para minimizar impactos; cuando fue posible, las pruebas se realizaron en momentos de baja carga. Se definieron periodos de retención y eliminación segura de datos conforme a buenas prácticas académicas y a la normativa aplicable.

Procedimiento

Esta sección describe el diseño del prototipo de software y su implementación, correspondiente a las Fases 3, 4 y 5 de la metodología IBD. Se profundiza en el diseño arquitectónico, el desarrollo de los componentes y la implementación en entorno real.

Para una mejor comprensión metodológica, se detallan siguiendo la estructura en 3 Etapas:

- 1) Planificación.
- 2) Desarrollo.
- 3) Validación y Despliegue.

Este flujo garantizó una transición estructurada desde la conceptualización técnica hasta la evaluación del sistema en un entorno controlado (*staging*).

ETAPA 1: Planificación

Diseño de la arquitectura del sistema

Stack tecnológico. Esta etapa se dedicó a la definición de los cimientos tecnológicos y estructurales de la plataforma. A partir de los requisitos funcionales y no funcionales levantados, se

realizó un análisis comparativo de tecnologías, frameworks y herramientas para cada capa del sistema. La selección final se basó en criterios de productividad del desarrollador, mantenibilidad a largo plazo, rendimiento, soporte comunitario y adecuación al contexto de las PYMES (recursos limitados, necesidad de soluciones ligeras). En la Tabla 5 se resumen las decisiones por capa o componente: tecnología elegida, alternativas evaluadas y criterio principal de selección. La tabla permite justificar de forma compacta la adopción de NestJS en el backend, PostgreSQL en persistencia, React Native + Expo en móvil, Next.js en panel web, JWT en autenticación y Docker en contenedorización, así como VPS (Hetzner) y Vercel para despliegue.

Tabla 5

Selección tecnológica para la arquitectura de la plataforma SaaS

Capa / Componente	Tecnología seleccionada	Alternativas evaluadas	Criterio de selección principal
Backend (API y lógica)	NestJS (No-de.js/TypeScript)	Express.js (Node.js), Django (Python), Spring Boot (Java)	Arquitectura modular por defecto, tipado estático (TypeScript) para reducir errores, inyección de dependencias integrada y ecosistema robusto para APIs escalables.
Base de datos	PostgreSQL	MySQL, MongoDB (NoSQL)	Modelo relacional adecuado para la integridad de datos transaccionales (pedidos, inventario). Balance entre rendimiento, características avanzadas y costo cero (open-source).

Capa / Componente	Tecnología seleccionada	Alternativas evaluadas	Criterio de selección principal
Aplicación móvil	React Native + Expo	Flutter (Dart), desarrollo nativo (Kotlin/Swift)	Desarrollo ágil y eficiente para Android. Expo agiliza el ciclo de desarrollo (builds, testing) y el acceso a APIs nativas sin complejidad.
Panel web admin	Next.js (React) + Tailwind CSS	React (Create React App), Vue.js, Angular	Renderizado híbrido (SSR/CSR) de Next.js para mejor SEO y performance de carga; Tailwind CSS para estilizado ágil y consistente.
Autenticación	JWT (JSON Web Tokens) + bcrypt	Sesiones en servidor, OAuth 2.0 (third-party)	Stateless y escalable para APIs REST. Simplicidad de implementación y seguridad suficiente para el alcance del piloto.
Contenedorización	Docker	–	Estandarización del entorno de desarrollo y despliegue, con consistencia entre equipos y facilidad para desplegar en la nube.
Despliegue backend/DB	VPS (Hetzner)	AWS/Azure (cloud), Railway, Render	Costo-eficiencia y control total del servidor para un piloto, con rendimiento predecible y facturación simple.

Capa / Componente	Tecnología seleccionada	Alternativas evaluadas	Criterio de selección principal
Despliegue panel web	Vercel	Netlify, GitHub Pages	Integración nativa con Next.js, despliegue automático por Git, CDN global y SSL gratuito, minimizando tareas de DevOps.

Arquitectura de la Plataforma

Una vez establecidas las herramientas tecnológicas, se definió una arquitectura principal **modular en capas**: modular por dominios de negocio y en capas para separar responsabilidades técnicas. Los componentes principales de la plataforma son:

- Aplicación móvil para clientes, repartidores y vista de cocina (aceptar o rechazar pedidos, notificar cuando esté listo para que el repartidor recoja, integrado al flujo de órdenes).
- Aplicación web principalmente para administración, con posibilidad de iniciar sesión como cliente para realizar compras desde la web.
- Servicios backends con sus respectivas APIs.

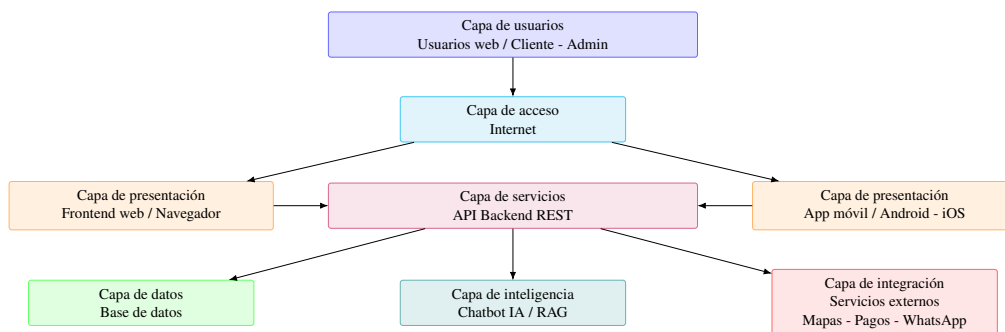


Figura 3
Arquitectura por capas de la plataforma SaaS

La arquitectura está organizada en cuatro capas principales: presentación (aplicación móvil y web), lógica de negocio (backend), persistencia (base de datos) e infraestructura (despliegue en nube). Cada capa se comunica con las adyacentes mediante interfaces bien definidas, mientras que la separación por módulos permite evolucionar funcionalidades sin afectar dominios no relacionados. Como parte de la Etapa 1, se diseñó también el flujo integral del sistema para modelar de extremo a extremo la interacción cliente-cocina-repartidor-dashboard; dicho flujo se detalla en la Figura 11 y se implementa en la Etapa 2. Como material complementario, la versión ampliada del diagrama por capas se incluye en el Apéndice B.

Arquitectura de la aplicación móvil

La aplicación móvil es la interfaz principal para tres roles: cliente, repartidor y cocina. Utiliza React Native + Expo para maximizar la reutilización de código entre plataformas. La versión ampliada de esta arquitectura se presenta en el Apéndice C.

Arquitectura de la aplicación web

El panel web administrativo permite la gestión de productos, categorías, pedidos y repartidores, además de proporcionar acceso a clientes para realizar compras. La versión ampliada del diagrama del panel web se presenta en el Apéndice C.

Arquitectura del backend

El backend expone una API RESTful que centraliza toda la lógica de negocio, autenticación y manejo de datos. La versión ampliada de la arquitectura modular del backend se incluye en el Apéndice D.

Esta figura mantiene la misma escala visual que las arquitecturas móvil y web para asegurar uniformidad de presentación y facilitar la comparación entre componentes.

Arquitectura de la aplicación móvil

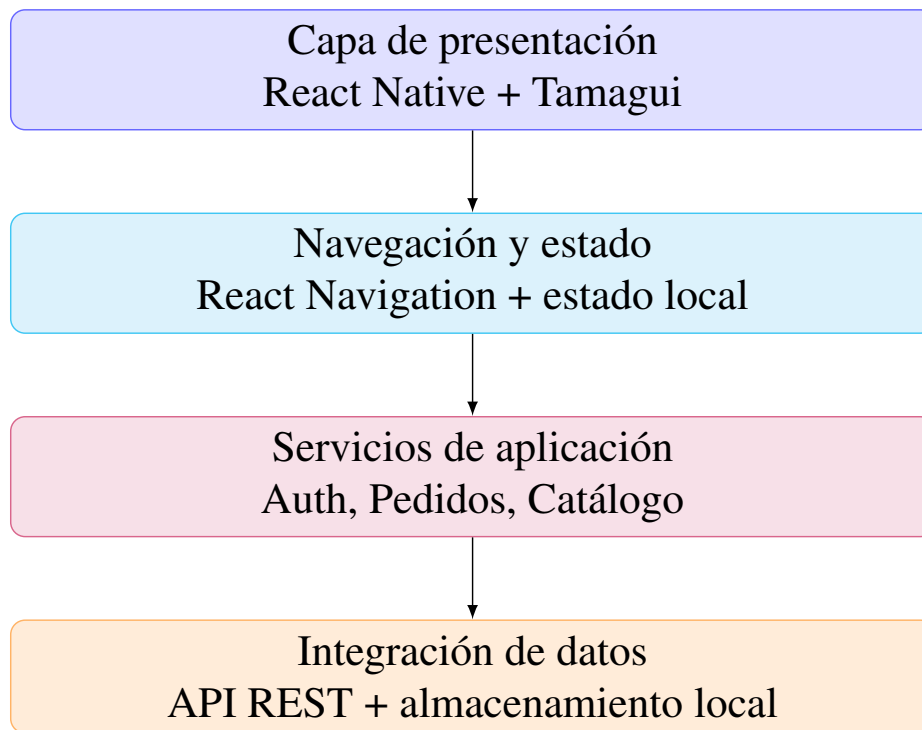


Figura 4

Arquitectura de la aplicación móvil

Arquitectura del panel web

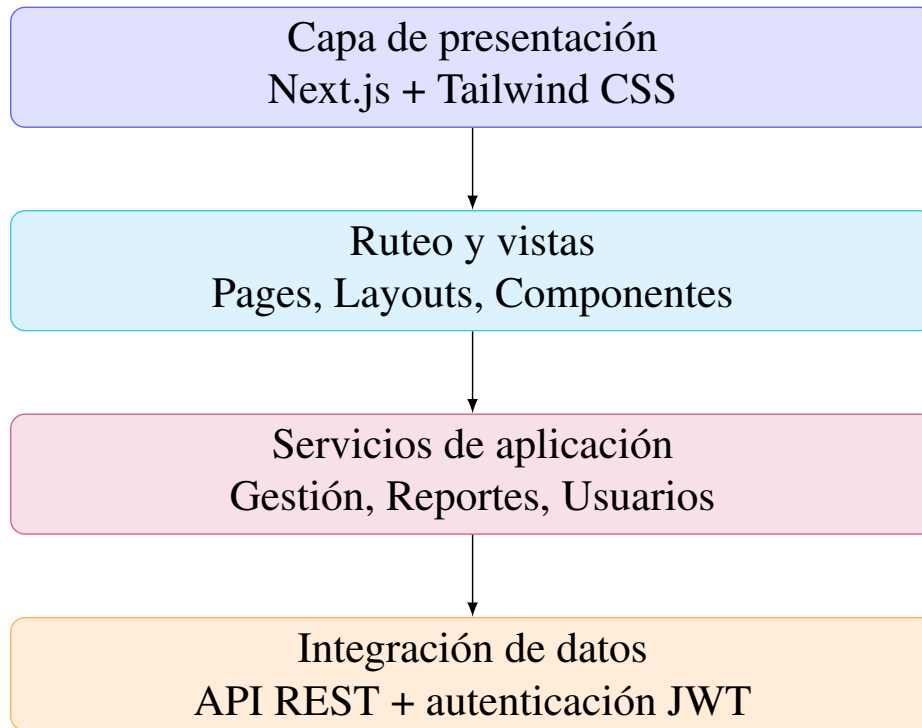


Figura 5
Arquitectura de la aplicación web

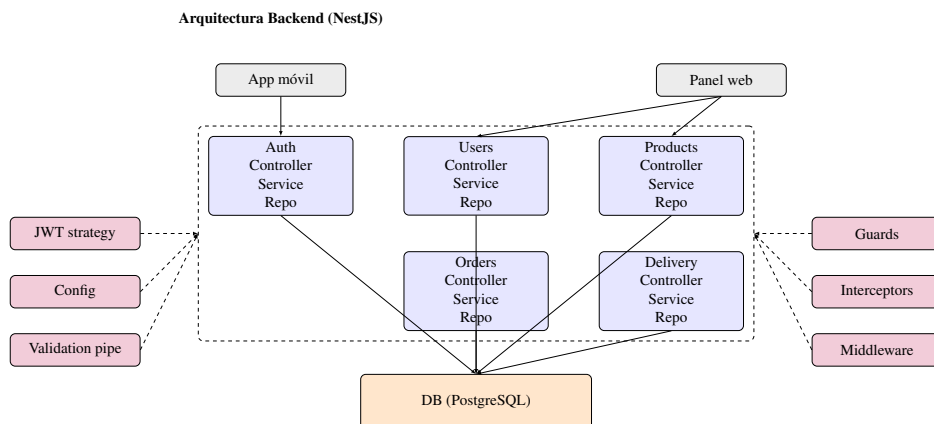


Figura 6
Arquitectura del backend (API REST)

Arquitectura de la base de datos

Objetivo del modelo de datos: garantizar integridad referencial, consultas eficientes y soporte para las operaciones del negocio mediante un esquema relacional estructurado.

Para esta plataforma se utilizó un único modelo de datos: **relacional**, implementado con PostgreSQL. No se emplearon bases de datos NoSQL; la elección de un sistema de gestión de bases de datos relacionales (RDBMS) se debió a la necesidad de integridad referencial (asegurar que un pedido refiere a un usuario existente, que un producto pertenece a una categoría válida), a la facilidad para realizar uniones (JOINS) entre tablas y a la adecuación del modelo relacional para transacciones (pedidos, inventario) y consultas complejas (informes, métricas). Se evaluaron alternativas como MongoDB (NoSQL) y MySQL; se descartaron por priorizar consistencia transaccional y esquema estructurado. La implementación se realizó mediante Supabase como proveedor de PostgreSQL administrado (PaaS). La opción de una base de datos en la nube bajo modelo PaaS permite delegar backups, escalado y mantenimiento del motor, reduciendo la carga operativa del equipo y alineándose con el contexto de recursos limitados de las PYMES objetivo.

Modelo Entidad-Relación principal: A grandes rasgos, se definen las siguientes entidades (tablas) en el esquema de datos.

Usuario (tbl_usuarios): Representa a los usuarios de la plataforma. Incluye id (clave primaria), nombre, correo electrónico, contraseña encriptada con bcrypt, teléfono y fecha de registro.

Implementa un sistema de roles mediante relación muchos a muchos con la entidad Rol, permitiendo que un usuario tenga múltiples roles simultáneamente. Para usuarios con rol de repartidor, incluye campos específicos como imagen de perfil, descripción, tipo de vehículo y un estado de aprobación (pendiente, aprobado, rechazado) gestionado por administradores. Los usuarios se asocian a una ciudad mediante clave foránea.

Rol (tbl_roles): Define los roles del sistema (cliente, repartidor, admin, superadmin). Se relaciona con usuarios mediante tabla intermedia, soportando asignación flexible de roles. Los roles se emplean en el sistema de autorización basado en RBAC (Role-Based Access Control).

Producto (tbl_productos): Representa los productos disponibles para venta. Contiene id, nombre, descripción, imagen (URL), categoría (referencia textual), precio y stock global. Incluye un campo activo para control de disponibilidad. Se relaciona con items de pedido (OrderItem) y con stock por local (StockLocal) para control de inventario distribuido.

Categoría (tbl_categorias): Agrupa productos por tipo. Incluye id, nombre, descripción, imagen_url, ícono, color (para UI), estado activo y orden de presentación. No se vincula directamente con productos mediante clave foránea sino por referencia textual en el campo categoría del producto.

Pedido (tbl_pedidos): Registra cada orden realizada por un cliente. Contiene id, fecha del pedido, estado (pendiente, en_camino, entregado, cancelado), total, tiempo de entrega estimado, referencia al cliente (id_usuario), dirección de entrega (id_direccion) y repartidor asignado (id_repartidor, nullable). Se relaciona con items de pedido (OrderItem) que detallan los productos incluidos.

OrderItem (detalle de pedido): Tabla de relación muchos a muchos entre pedidos y productos. Almacena la cantidad de cada producto en un pedido específico, el precio unitario al momento de la compra y subtotales, manteniendo histórico de precios independiente de cambios futuros.

Ciudad (tbl_ciudades): Representa las ciudades donde opera el servicio. Contiene id y nombre. Usuarios y locales se asocian a ciudades, permitiendo segmentación geográfica de operaciones.

Local/Tienda (tbl_locales): Representa puntos de venta físicos. Incluye id, nombre, dirección, teléfono, descripción, estado activo, horarios de apertura y cierre, coordenadas geográficas (latitud/longitud) para mapas, y relación con ciudad. Se relaciona con direcciones de clientes y stock local de productos.

Dirección (tbl_direcciones): Almacena direcciones de entrega de clientes. Contiene id, usuario propietario, local asociado (opcional), calle principal, calle secundaria, número de casa, referencia, coordenadas geográficas y ciudad. Permite múltiples direcciones por usuario.

StockLocal: Gestiona inventario por local. Relaciona productos con locales y almacena la cantidad

disponible en cada punto de venta, habilitando control de stock distribuido.

Además de las relaciones, se definieron índices en campos de búsqueda frecuente (por ejemplo, estado y fecha en pedidos, correo en usuarios) para optimizar consultas, y restricciones de unicidad en claves naturales donde corresponde (emails de usuarios, nombres de roles). Se cuidó la normalización para evitar duplicidades innecesarias, manteniendo un equilibrio pragmático entre integridad y rendimiento. En entidades con alto volumen de escritura (pedidos y sus items) se contempló el uso de transacciones para asegurar consistencia en operaciones compuestas.

Implementación con TypeORM

En la implementación con TypeORM, cada entidad se refleja en una clase TypeScript con decoradores `@Entity` que mapean a las tablas correspondientes. Las relaciones se definen mediante decoradores especializados:

- `@ManyToOne` y `@OneToMany` para relaciones uno a muchos (ej: Usuario-Pedidos, Pedido-OrderItems)
- `@ManyToMany` con `@JoinTable` para relaciones muchos a muchos (ej: Usuario-Roles)
- `@JoinColumn` para especificar claves foráneas explícitas

Por ejemplo, la entidad Order (Pedido) define `@ManyToOne(() => User, user => user.orders)` para referenciar al cliente, con un `@JoinColumn` indicando `id_usuario` como clave foránea. De forma similar, referencia al repartidor asignado mediante otra relación a User (nullable), distinguiendo roles mediante el sistema RBAC implementado.

La configuración de TypeORM permite sincronización automática del esquema en desarrollo (`synchronize: true` cuando `NODE_ENV !== 'production'`), facilitando iteraciones rápidas. Las entidades emplean transformadores personalizados para tipos específicos (p. ej., convertir strings decimales de PostgreSQL a números JavaScript en campos de precio).

Integridad referencial y reglas de negocio

Se definieron claves foráneas para garantizar consistencia: un pedido siempre referencia un usuario y dirección válidos. Para mantener integridad histórica, no se permite eliminación física de usuarios con pedidos; en su lugar se emplea un campo activo o estado para desactivación lógica. Las relaciones críticas previenen borrados cascada no deseados, protegiendo el histórico transaccional.

La arquitectura soporta operaciones distribuidas por ciudad y local. Aunque actualmente opera para un cliente piloto, la estructura con entidades Ciudad y Local sienta las bases para expansión geográfica. El control de stock por local (StockLocal) permite gestión descentralizada de inventario, relevante para cadenas o franquicias futuras.

Estrategia de escalabilidad

Si bien la implementación actual emplea una única base de datos para el cliente piloto, la arquitectura contempla escalabilidad futura mediante dos enfoques posibles:

- 1) **Base de datos por cliente:** Estrategia de multi-tenancy mediante bases de datos separadas por cliente empresarial. NestJS/TypeORM soportan conexiones dinámicas, permitiendo que cada empresa opere en su propia instancia PostgreSQL, garantizando aislamiento total de datos y cumplimiento de normativas de privacidad.
- 2) **Segmentación geográfica:** Escalar horizontalmente mediante réplicas por región geográfica, aprovechando la entidad Ciudad para enrutamiento de consultas.

La configuración actual con Supabase facilita esta migración, al soportar múltiples proyectos de base de datos administrados de forma independiente.

Finalmente, se contemplaron estrategias de respaldo automático (backups diarios mediante Supabase), recuperación ante desastres y mecanismos de migración de esquema controlada mediante archivos SQL versionados (ver carpeta migrations/ con ejemplos como create-stock-local-table.sql), abandonando sincronización automática en producción. La versión

ampliada del diagrama entidad-relación también se incorpora en el Apéndice A.

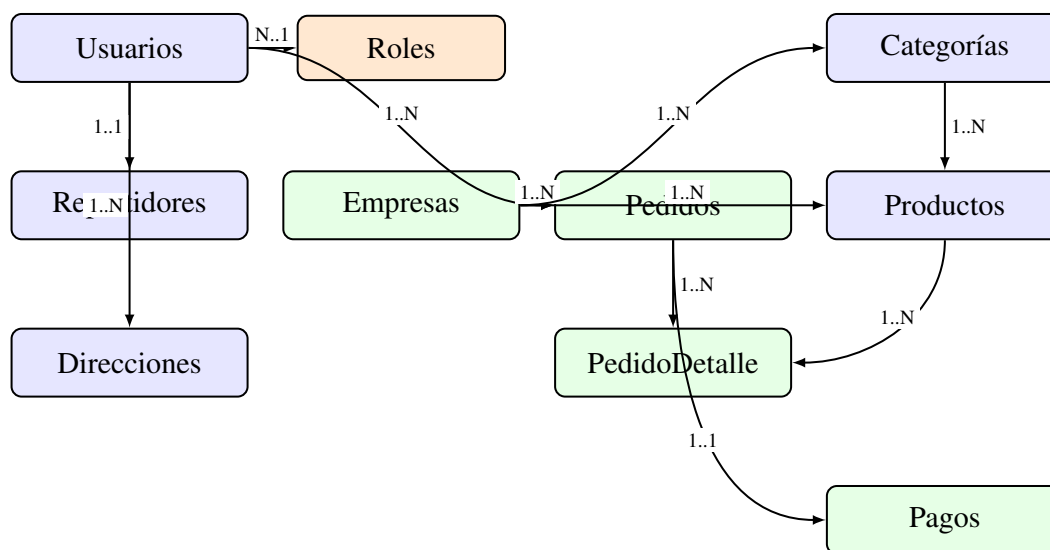


Figura 7
Diagrama Entidad-Relación (ER) de la base de datos

Roles, permisos y seguridad (JWT, RBAC)

La plataforma implementa un esquema de roles y permisos (RBAC) para garantizar que cada usuario solo realice las acciones que le corresponden (Logto, s.f.). RBAC gestiona "quién puede hacer qué" agrupando permisos en roles; a cada usuario se le asigna uno o varios roles y cada rol confiere permisos sobre funciones, APIs o datos.

Administrador (ADMIN): Usuario del panel web con privilegios completos. Puede crear y editar productos y categorías, ver todos los pedidos, gestionar repartidores (aprobar/rechazar) y configurar la plataforma. Existe un rol adicional de Superadmin con acceso irrestricto al sistema completo.

Cliente (CLIENTE): Usuario de la app móvil que realiza pedidos. Puede registrarse o iniciar sesión, ver y buscar productos, crear pedidos, consultar su estado y editar su perfil.

Repartidor (DELIVERY): Usuario de la app móvil que toma y entrega pedidos. Puede ver pedidos pendientes, aceptar/declinar encargos, marcar entregas y revisar su historial. Requiere aprobación administrativa antes de operar.

Cocina (COCINA): Usuario de la app móvil (vista de cocina) encargado de la gestión de pedidos. Recibe las órdenes nuevas y tiene la capacidad de aceptarlas o rechazarlas; una vez preparadas, marca el pedido como listo para que el repartidor proceda a recogerlo y entregarlo al cliente, actualizando el estado del pedido en el flujo del sistema.

Estos roles se implementan mediante una tabla independiente (tbl_roles) y una relación muchos a muchos con usuarios, permitiendo que un mismo usuario posea múltiples roles simultáneamente. La tabla de roles almacena los identificadores estándar (cliente, repartidor, admin, superadmin) con restricción de unicidad. Esta arquitectura flexible facilita la gestión de permisos compuestos; por ejemplo, un usuario puede ser simultáneamente cliente y administrador.

La autorización por roles se implementa de forma declarativa en endpoints críticos, con mensajes de error consistentes y registro de intentos no autorizados para auditoría. En cuanto a autenticación, se definieron políticas de expiración de tokens equilibradas entre seguridad y usabilidad; por simplicidad, inicialmente no se emplean tokens de actualización (refresh), aunque se prevé su inclusión si el uso lo amerita. Se recomienda almacenar credenciales y tokens de forma segura en cliente (cookies HttpOnly en web, almacenamiento seguro en móvil) y evitar exposición en logs.

Para la autenticación y autorización se utilizó JWT (JSON Web Tokens) como mecanismo stateless. JWT es un estándar abierto (RFC 7519) para transmitir información de forma segura en forma de token JSON firmado y se ha convertido en estándar de facto para proteger endpoints sensibles de APIs (Aguilera, s.f.). La implementación se realizó mediante la integración de Passport.js en NestJS con la estrategia passport-jwt. El detalle gráfico extendido del flujo de autenticación y autorización se presenta en el Apéndice E.

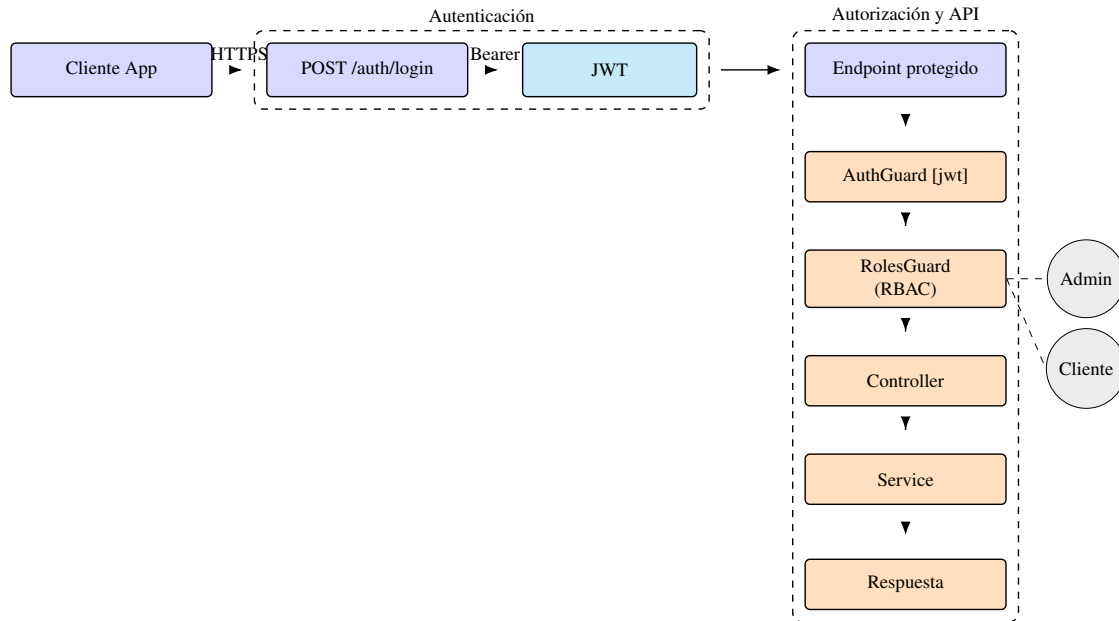


Figura 8
Flujo de autenticación y autorización (JWT + RBAC)

ETAPA 2: Desarrollo

Desarrollo de la Plataforma SaaS

Los Objetivos Específicos 3, 4 y 5 se centraron en la construcción iterativa e integración de todos los componentes del sistema. Se siguió una metodología ágil, organizando el trabajo en sprints de dos semanas y consolidando evidencia de avance en la bitácora técnica de validación (Apéndice J). El desarrollo fue modular y paralelo. Configuración inicial: se establecieron los tres repositorios (backend, móvil, web), se configuraron las herramientas de contenedorización (Docker) y se definió la estructura base de la base de datos en PostgreSQL.

Desarrollo del frontend móvil (React Native + Expo)

Objetivo del frontend: ofrecer una experiencia clara y eficiente para clientes, repartidores y personal de cocina, con navegación predecible, validaciones tempranas y comunicación robusta con la API. El frontend móvil se implementó con React Native y Expo, lo que permite desarrollar una

aplicación nativa multiplataforma a partir de una sola base de código.

React Native facilita la reutilización entre iOS y Android sin comprometer el rendimiento; Expo simplifica el ciclo de vida de la aplicación al ofrecer herramientas integradas para compilar, desplegar y probar sin configurar proyectos nativos desde cero (campusMVP, s.f.). Además, Expo expone funcionalidades nativas desde JavaScript (cámara, notificaciones, sensores) y habilita un flujo ágil con recarga en vivo y publicación mediante enlaces o códigos QR, favoreciendo la colaboración y las pruebas (campusMVP, s.f.).

La aplicación ofrece flujos diferenciados para tres perfiles: Cliente, Repartidor y Cocina, además de pantallas de inicio de sesión y registro. Se incorporó un módulo o vista de *cocina* en la aplicación móvil que permite aceptar o rechazar pedidos y notificar cuando el pedido está listo para que el repartidor recoja, conectado al flujo de solicitud de órdenes. Así, cuando un cliente realiza un pedido, la cocina puede visualizarlo, aceptarlo o rechazarlo, y al marcarlo como listo el repartidor puede ir a recogerlo para la entrega. La navegación se gestiona con la librería de React Native (p. ej., React Navigation), que permite transicionar entre vistas, mantener historiales y pasar parámetros. Cada rol visualiza únicamente las opciones pertinentes; esta segregación mejora la experiencia y refuerza la seguridad, complementada con validaciones en el backend. El flujo visual ampliado por rol se documenta en el Apéndice F.

La arquitectura del frontend móvil privilegia la simplicidad y la previsibilidad del estado. Se emplea estado local y patrones de elevación de estado donde corresponde, con manejo de formularios controlados y validación básica en cliente (tipos de entrada, obligatoriedad, formatos). La comunicación con la API se realiza mediante solicitudes HTTP que envían y reciben JSON; se contemplan casos de error (credenciales inválidas, falta de conectividad, respuestas 4xx/5xx) con mensajes claros al usuario y estrategias de reintento cuando es pertinente. La interfaz se diseñó con principios de consistencia visual y accesibilidad, cuidando contrastes, tamaños de toque y retroalimentación tras acciones. Para el diseño visual se empleó Tamagui, biblioteca de UI y sistema de estilos compatible con React Native, que provee componentes y temas para construir

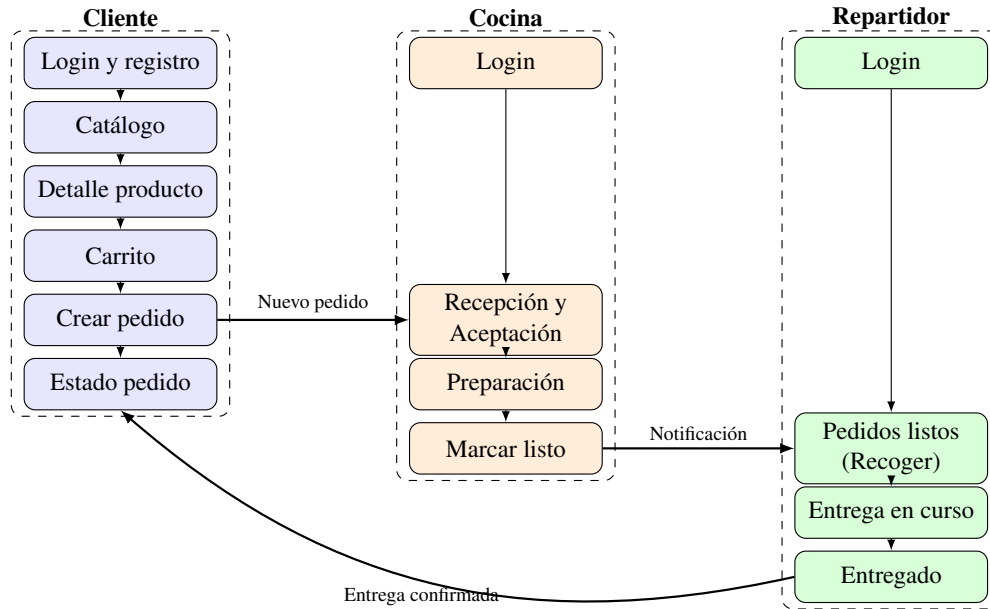


Figura 9

Flujo de la interfaz móvil por rol (cliente, repartidor, cocina)

interfaces coherentes y responsivas (Tamagui, s.f.). En el proyecto se estilizaron botones, tarjetas y formularios con componentes Tamagui, logrando consistencia y mantenibilidad. Su sistema de temas facilita la personalización visual (colores, tipografía) y asegura una apariencia uniforme en la aplicación Android, adaptándose a las necesidades de identidad del negocio.

Desarrollo del frontend panel web (Next.js + Tailwind CSS)

Objetivo del panel web: proveer principalmente herramientas de administración para gestionar catálogo, pedidos y repartidores, con formularios validados, vistas responsivas y flujos CRUD claros; además, se permite iniciar sesión como cliente para realizar compras desde la web.

Se desarrolló un sitio web con React y Next.js cuya función principal es la administración del negocio (panel administrativo). También se implementó la posibilidad de que los clientes inicien sesión desde la web para realizar pedidos desde el navegador, complementando la opción de la aplicación móvil. Se aprovechó el renderizado del lado del servidor (SSR) en vistas específicas de dashboard cuando resultó conveniente.

El estilado se realizó con Tailwind CSS, framework utility-first que permite componer diseños rápidos y consistentes mediante clases utilitarias (Tailwind CSS, s.f.). Su configuración admite alta personalización, útil para adaptar paletas o temas por cliente.

En el panel se aplicaron formularios validados con reglas de negocio simples (campos requeridos, formatos, rangos) y se implementaron flujos habituales de CRUD con confirmaciones y mensajes de éxito/error.

La estructura de navegación organiza secciones por dominio (productos, categorías, pedidos, repartidores) y proporciona filtros básicos (por estado, fecha) para facilitar la gestión. Se cuidó la responsividad para operar en distintos tamaños de pantalla, y se diseñaron tablas y listas con paginación prevista para escenarios de datos voluminosos.

El panel permite crear y editar productos, gestionar categorías y aprobar o rechazar repartidores que solicitan unirse. Los formularios incorporan validaciones de campos; las vistas de lista presentan pedidos en curso y solicitudes pendientes.

Cuando un repartidor se registra desde la app móvil, su cuenta queda "pendiente" hasta la revisión administrativa, tras la cual puede ser habilitada o rechazada, garantizando control de calidad. El flujo visual ampliado del panel web se documenta en el Apéndice F.

Ambos frontends consumen el mismo backend mediante API REST y JSON. La separación de responsabilidades mantiene el frontend enfocado en la experiencia y presentación, mientras la lógica de negocio y la persistencia residen en el backend.

En materia de rendimiento y experiencia, se optimizaron listas y vistas frecuentes y se cuidó la retroalimentación visual en operaciones que pueden tardar (carga de catálogo, envío de pedidos). Se delinearon buenas prácticas para el manejo de imágenes (compresión, tamaños adecuados) y para el control de estados vacíos y errores de red, mejorando la robustez de la interfaz en condiciones reales.

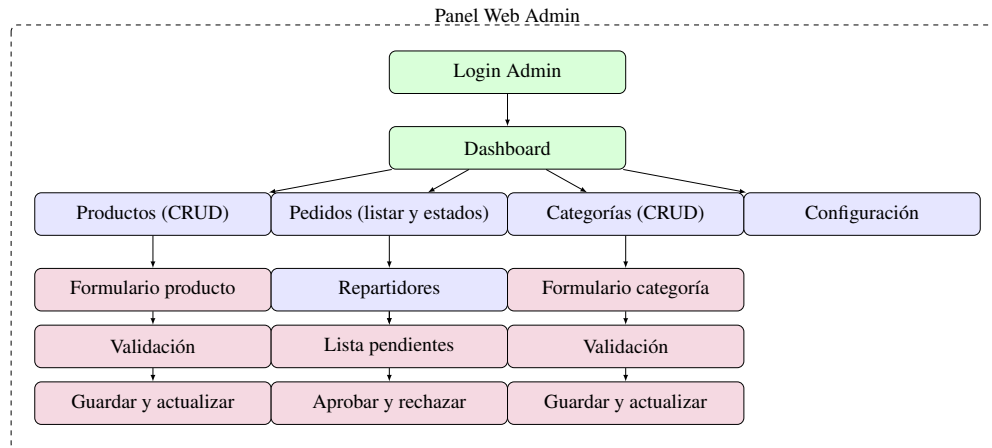


Figura 10
Flujo del panel web administrativo

Desarrollo del backend (NestJS, capas, módulos)

Objetivo del backend: exponer servicios estables y seguros, aplicar reglas de negocio y coordinar persistencia con una arquitectura modular mantenible.

Decisión arquitectónica

La arquitectura del backend se definió como **modular en capas**. Se implementó con NestJS, framework de Node.js que aporta estructura modular nativa basada en TypeScript, decoradores e inyección de dependencias. Frente a Express puro, NestJS ofrece una organización más clara para separar responsabilidades, facilitar la escalabilidad y mantener consistencia en proyectos con múltiples dominios (usuarios, productos, pedidos y autenticación).

Estructura interna por capas

La arquitectura técnica se organiza en tres capas principales: controlador, servicio y acceso a datos. El controlador expone endpoints HTTP y orquesta el flujo de entrada; el servicio concentra reglas de negocio; y el repositorio o capa de datos persiste y consulta información en PostgreSQL mediante TypeORM. Esta separación mantiene un flujo unidireccional (controlador → servicio → datos), evita acoplamiento entre transporte y lógica, y simplifica el mantenimiento evolutivo.

Componentes transversales

Se definieron DTOs para tipado y validación de entrada/salida; la integración de class-validator y class-transformer con ValidationPipe global permite rechazar solicitudes mal formadas antes de ejecutar lógica de negocio. La inyección de dependencias con @Injectable() mantiene el código desacoplado y testeable. En persistencia, TypeORM se integró con @nestjs/typeorm, utilizando entidades tipadas, repositorios y QueryBuilder para operaciones transaccionales y consultas estructuradas.

Módulos de dominio

La organización del backend combina capas técnicas y módulos de dominio. Cada módulo agrupa su controlador, servicios y acceso a datos en torno a una responsabilidad funcional, lo que permite describir con claridad su función e importancia en el sistema. Esta organización mejora la trazabilidad entre requisito, endpoint e implementación.

La organización por módulos comprende:

- **AuthModule:** Gestiona registro, login y emisión de tokens JWT. Emplea bcrypt para hash de contraseñas (salt rounds: 10) y JwtModule configurado con clave secreta y expiración. Implementa estrategia Passport-JWT para validar tokens en headers Authorization.
- **UsersModule:** Administra usuarios y relación con roles. Provee endpoints protegidos (solo admin/superadmin) para CRUD de usuarios, gestión de roles y consulta de perfiles. Integrado estrechamente con AuthModule.
- **ProductsModule:** Gestión de catálogo de productos. Endpoints públicos para listar y consultar productos; endpoints protegidos (admin/superadmin) para crear, actualizar y eliminar. Integra StockLocal para control de inventario por local.
- **CategoriesModule:** CRUD de categorías de productos. Endpoints protegidos por roles, con listado público para clientes.

- **OrdersModule:** Creación, consulta y gestión de estados de pedidos. Clientes crean pedidos; repartidores consultan asignaciones; admins visualizan todos los pedidos con filtros por estado. Maneja relaciones complejas con OrderItems, Users, Addresses y Products.
- **AddressesModule:** Gestión de direcciones de entrega por usuario. Cada usuario maneja sus propias direcciones; validación de pertenencia mediante guards.
- **CitiesModule:** CRUD de ciudades. Listado público; creación/eliminación restringida a admin/superadmin.
- **StoresModule:** Gestión de locales comerciales con información geográfica (coordenadas, horarios). Usado para asignación de pedidos y control de stock distribuido.
- **AdminModule:** Operaciones administrativas específicas como aprobación/rechazo de repartidores, reportes y configuraciones de negocio.

NestJS se integra con Passport.js para la autenticación; se definió una estrategia JWT personalizada (JwtStrategy) que extrae y valida el token, decodifica el payload y adjunta el usuario al objeto request para uso en controladores y guards.

Los **Guards**, **Interceptors** y **Middleware** se aplican en puntos concretos del flujo de la petición. **JwtAuthGuard** se usa en todos los endpoints privados (p. ej. /api/orders, /api/products en escritura, /api/users, /api/addresses): valida el token JWT en el header Authorization y adjunta el usuario al request. **RolesGuard** se aplica junto con el decorador @Roles() en rutas que requieren rol específico: por ejemplo, /api/products (POST) y /api/categories (CRUD) exigen rol admin o superadmin; /api/orders para crear pedidos exige cliente; las rutas de repartidores exigen rol delivery. **ThrottlerModule** está configurado globalmente (100 peticiones por minuto por IP) para todas las rutas. **ValidationPipe** global se ejecuta en el bootstrap antes de que la petición llegue al controlador, aplicándose a todos los endpoints que reciben DTOs en el body.

Se contempla incorporar interceptores para logging de auditoría, medición de tiempos de respuesta y formateo consistente de respuestas JSON.

En validación de datos, se activó ValidationPipe global con opciones whitelist (elimina propiedades no declaradas en DTOs) y forbidNonWhitelisted (rechaza requests con campos extra), robusteciendo la API ante entradas malformadas y ataques de inyección de datos.

Integración de módulos funcionales: una vez establecida la comunicación básica entre frontend y backend vía API REST, se implementaron las funcionalidades específicas.

Módulo de Pedidos

El flujo general del proceso de pedido contempla la creación, estados (pendiente, en camino, entregado) y asociación con geolocalización. La Figura 11 ilustra la secuencia completa. A nivel de backend, el flujo de creación de un pedido es el siguiente: (1) el cliente autenticado envía POST a /api/orders con productos, cantidades y dirección; (2) JwtAuthGuard valida el token y adjunta el usuario al request; (3) ValidationPipe valida el DTO; (4) OrdersController delega en OrdersService.create(); (5) el servicio verifica usuario, dirección y stock; (6) se crea la entidad Order y los OrderItem en una transacción TypeORM; (7) se retorna 201 Created. Este flujo ejemplifica la separación de responsabilidades entre capas y la lógica de negocio encapsulada en el servicio. El diagrama ampliado de este flujo se incluye en el Apéndice G.

Módulo de Inventario:

Lógica para descontar stock y prevenir ventas de productos agotados. La versión ampliada del diagrama se presenta en el Apéndice G.

Módulo de Repartidores:

Proceso de registro, aprobación administrativa y asignación manual de pedidos. La versión ampliada del flujo se presenta en el Apéndice G.

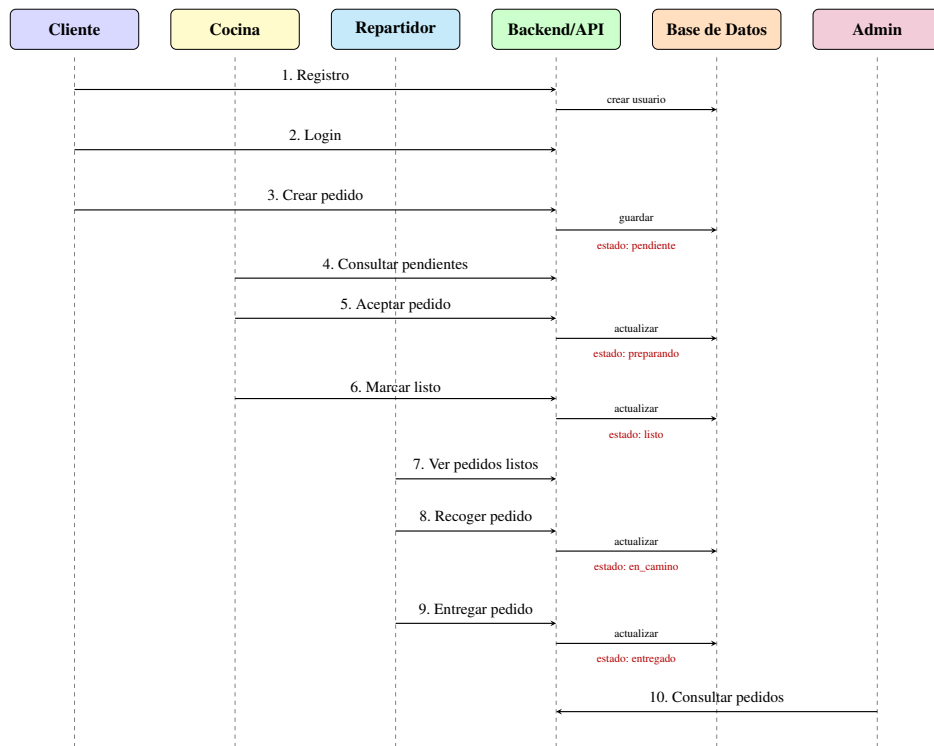


Figura 11
 Diagrama de secuencia: flujo completo del proceso de pedido. Fuente: elaboración propia.

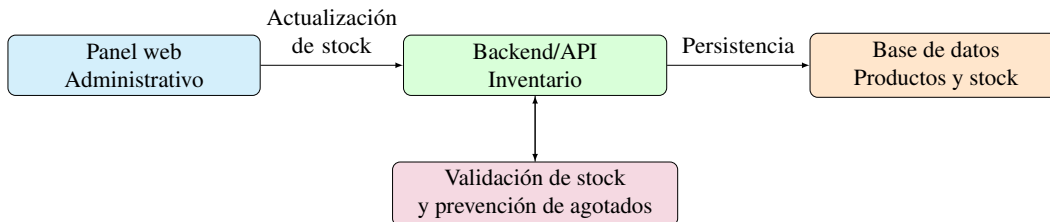


Figura 12
 Lógica del módulo de inventarios y control de stock

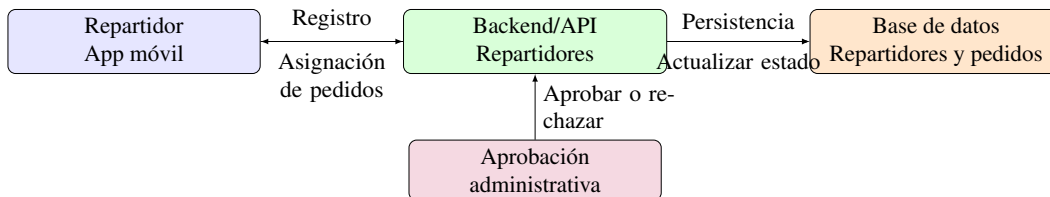


Figura 13
 Flujo del módulo de repartidores

Dashboard Analítico:

Consultas SQL y visualización de KPIs (total de pedidos, ventas por día).

Chatbot RAG por WhatsApp

El chatbot se implementó bajo el paradigma RAG (Retrieval-Augmented Generation): se recuperan fragmentos relevantes de una base de conocimiento y se inyectan en el prompt del modelo de lenguaje para generar respuestas contextualizadas. **Arquitectura:** servicio independiente (Node.js) que expone un webhook recibiendo mensajes entrantes de WhatsApp Business Cloud API; el flujo consiste en (1) recibir mensaje, (2) embedding de la consulta del usuario, (3) búsqueda por similitud en una base vectorial, (4) construcción del prompt con el contexto recuperado más la consulta, (5) llamada al LLM, (6) envío de la respuesta al usuario vía API de WhatsApp. **Modelo de lenguaje:** se utilizó un LLM accesible vía API (modelo tipo GPT/compatible con LangChain) para la generación de respuestas. **Base vectorial e indexación:** el corpus de la plataforma (menú, horarios, preguntas frecuentes, información de pedidos) se fragmentó en chunks, se generaron embeddings con el mismo modelo de embeddings del LLM y se almacenaron en una base vectorial (p. ej. en memoria o integrada en LangChain) para búsqueda por similitud semántica. La indexación se realizó de forma previa al despliegue y se puede actualizar cuando cambie el catálogo o las políticas. **Integración con WhatsApp:** se utilizó la API de WhatsApp Business Cloud (webhook configurado en el backend para recibir mensajes entrantes y enviar respuestas mediante el endpoint de mensajes de la API). **Validación:** se validó mediante pruebas con consultas frecuentes (horarios, menú, estado de pedido); se verificó que las respuestas fueran coherentes con el contexto recuperado y que el tiempo de respuesta fuera aceptable para uso conversacional. No se aplicaron métricas formales de evaluación (p. ej. BLEU o satisfacción) en este piloto; la validación fue cualitativa y operativa.

Clasificación de la solución como SaaS y alcance del piloto

La solución se clasifica como SaaS en la medida en que se entrega como **servicio alojado en la nube**, sin instalación en las instalaciones del cliente: el backend y la base de datos están en un VPS y en Supabase, el panel web en Vercel y la app móvil se distribuye vía enlace/APK con conexión a la misma API. El usuario no gestiona servidores ni actualizaciones del stack; el proveedor (en este caso el equipo del proyecto) centraliza el despliegue y las actualizaciones. **En este piloto no se implementó arquitectura multi-tenant** (una instancia por cliente); la solución opera como **prototipo aplicado a una PYME piloto**, con una única instancia de base de datos y backend. Para evolucionar hacia un modelo multiempresa típico de SaaS, sería necesario incorporar multi-tenancy (p. ej. base de datos por cliente o esquema por tenant), aislamiento de datos por organización, configuración independiente por cliente y un esquema de provisión y facturación; ello queda fuera del alcance del presente trabajo y se considera trabajo futuro.

Pruebas técnicas: Se realizaron pruebas unitarias en servicios críticos y pruebas de integración manuales para garantizar la correcta comunicación entre componentes.

ETAPA 3: Validación y Despliegue

Arquitectura del despliegue

Se configuró un VPS en Hetzner con Docker para alojar el backend y la conectividad segura hacia la base de datos PostgreSQL administrada en Supabase. El panel web se desplegó automáticamente en Vercel con integración continua por repositorio. La aplicación móvil se compiló y distribuyó como archivo APK para su instalación en dispositivos Android del local piloto. La infraestructura operó con separación funcional: capa de presentación (web y app), capa API (NestJS), capa de datos (PostgreSQL) y servicios externos (WhatsApp Business Cloud API), con comunicación por HTTPS y control de acceso basado en JWT. La versión ampliada de la arquitectura de despliegue se incorpora en el Apéndice H.

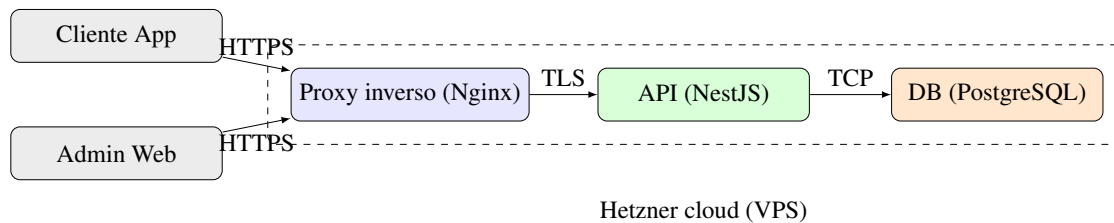


Figura 14
Arquitectura de despliegue (VPS, Vercel, APK)

Seguridad operativa y configuración de despliegue

Las medidas de seguridad y despliegue se consolidan en esta etapa porque se validan sobre el entorno *staging* y la infraestructura efectiva del piloto:

- **CORS:** Configurado por origen permitido, restringiendo acceso desde dominios no autorizados.
- **Rate Limiting:** ThrottlerModule con límite de 100 requests/minuto por IP.
- **Conexión SSL a base de datos:** Configuración de TypeORM con `ssl: true` y `rejectUnauthorized: false` para Supabase, con opciones de pool (`max: 20` clientes, `timeout: 30s`).
- **Variables de entorno:** ConfigModule global carga credenciales desde archivo `.env` (`DB_HOST`, `DB_PORT`, `DB_USERNAME`, `DB_PASSWORD`, `DB_NAME`, `JWT_SECRET`, `NODE_ENV`), evitando hard-coding de secretos.
- **Sincronización condicional:** TypeORM `synchronize` activo solo en desarrollo; en producción se emplean migraciones SQL versionadas.

El backend se despliega como aplicación Node.js independiente. Se contempló Docker para contenerización y reproducibilidad en VPS, con imágenes multi-stage para optimizar tamaño y perfiles por entorno (`development`, `production`). Para producción se planificó operación en VPS (Hetzner) con proceso administrado por PM2 o similar, proxy inverso Nginx para SSL/TLS y

monitoreo de salud del servicio. La base de datos PostgreSQL administrada por Supabase reduce la carga operativa de infraestructura.

Se recomienda un proxy inverso para gestionar certificados TLS y enrutar tráfico, así como automatizar despliegues con pipelines que ejecuten verificación y pruebas mínimas antes de publicar cambios. En resumen, el backend NestJS proporcionó una arquitectura modular robusta. Cada nueva funcionalidad se implementa añadiendo o modificando un módulo sin afectar otros, lo que agiliza el desarrollo colaborativo y reduce errores colaterales. La combinación de TypeScript, inyección de dependencias y la estructura en capas (Controlador-Servicio-Repositorio) hace que el código sea más fácil de seguir y depurar. NestJS, con su enfoque de convención sobre configuración, mantuvo aspectos como la seguridad, validación y estructura de proyecto cubiertos por diseño. El diagrama de red integral de interacción entre clientes, API y servicios externos se presenta en el Apéndice I.

Despliegue en *staging* y validación técnica

Esta etapa tuvo como propósito evaluar el sistema integral en entorno controlado (*staging*) mediante tareas guiadas y escenarios de prueba predefinidos. El procedimiento completo (despliegue, orientación, escenarios, tareas guiadas, recolección de datos y monitoreo técnico) se documenta en el Apéndice J. Para mantener este capítulo conciso, se resume el proceso en tres acciones clave:

- **Despliegue controlado:** Backend y base de datos en VPS con Docker; panel web en Vercel; app móvil en APK para pruebas.
- **Ejecución de escenarios:** Usuarios de prueba ejecutaron flujos críticos (pedido, cocina, reparto, administración y chatbot).
- **Registro de métricas:** Se capturaron tiempos, errores, completitud de tareas y logs técnicos para análisis posterior.

El detalle de las tareas evaluadas y sus criterios de éxito se presenta en el Apéndice J (Tabla 12).

Resultados

Introducción

El presente capítulo expone los resultados obtenidos de la investigación en torno a la digitalización de las PYMES gastronómicas de la ciudad de Portoviejo y a la validación técnica y de usabilidad de la plataforma SaaS desarrollada. Se estructura en tres apartados principales: (1) resultados del diagnóstico de digitalización previo a la implementación; (2) resultados de la validación de usabilidad y adopción; y (3) resultados técnicos del software, incluyendo pruebas, métricas de disponibilidad y rendimiento. La exposición es objetiva y descriptiva; las interpretaciones y relaciones con la literatura se abordan en el capítulo de Discusión.

La plataforma validada está constituida por tres componentes integrados: (1) un **backend** (API REST con NestJS y PostgreSQL) que centraliza autenticación por roles (administrador, cocina, repartidor, cliente), gestión de productos, categorías, pedidos, direcciones, locales y repartidores; (2) un **panel web** (Next.js) que permite a los administradores gestionar productos, categorías, pedidos, repartidores y reportes, y a los clientes realizar pedidos desde la tienda en línea (carrito, checkout con pago en efectivo o transferencia, historial de pedidos y direcciones); (3) una **aplicación móvil** (Expo/React Native) con vista específica para cocina (listar pedidos por ciudad, confirmar y preparar, marcar listo para recoger), flujo para repartidores (pedidos disponibles y asignados, aceptar pedido, marcar recogido, en camino y entregado, estado activo/inactivo) y flujo para clientes (productos por local, carrito, checkout, seguimiento del pedido). Los tres componentes consumen la misma API y fueron utilizados de forma conjunta por la PYME piloto durante el período de validación.

La presentación de resultados mantiene trazabilidad directa con los objetivos de investigación:

primero se reporta el diagnóstico que sustenta los requerimientos (Objetivo específico 1), luego la validación de usabilidad y métricas de desempeño (Objetivos específicos 2 y 5), y finalmente la evidencia técnica de pruebas unitarias/integración y validación funcional del artefacto (Objetivos específicos 3 y 4).

Resultados del diagnóstico de digitalización

En esta sección se presentan los resultados del análisis del estado de digitalización de la PYME gastronómica piloto antes de la implementación de la plataforma, identificando el nivel de adopción tecnológica, procesos operativos vigentes y brechas detectadas.

Caracterización de la PYME piloto

La PYME piloto seleccionada corresponde a un establecimiento gastronómico formalmente constituido en la ciudad de Portoviejo, con 3 años de operación, servicio de consumo en local y delivery. El negocio opera con 1 local físico, empleando a 8 personas: 1 administrador/propietario, 3 personas en cocina, 2 en atención al cliente y 2 repartidores. Atiende un promedio de 45-60 pedidos diarios, de los cuales aproximadamente el 65 % son delivery y 35 % consumo en local.

Estado de digitalización previo

La entrevista semiestructurada con el responsable del negocio (guía en Apéndice K) y la observación de procesos operativos revelaron el estado de digitalización previo. Las cantidades numéricas se obtuvieron a partir del registro de pedidos y canales reportados por el establecimiento durante un período de referencia de 2 semanas previas al diagnóstico (número total de pedidos registrados en ese período para calcular porcentajes). En la Tabla 6 se presenta el ítem, el número de eventos o observaciones que sustentan el porcentaje y el porcentaje correspondiente.

Nota. N.º eventos = número de pedidos o observaciones en el período de referencia de 2 semanas

Tabla 6*Estado de digitalización previo: eventos y porcentajes por ítem*

Ítem	N.º eventos / base	Porcentaje / descripción
Gestión de pedidos (canal)	127 pedidos (2 sem)	Teléfono 40 %; WhatsApp 45 %; presencial 15 %. Sin sistema centralizado; tiempo 5–7 min/pedido; errores $\approx$ 18 %.
Comunicación con cocina	Observación directa	Verbal/notas manuscritas; sin visibilidad de cola; retrasos 12–15 min en picos.
Gestión de repartidores	127 asignaciones	100 % manual (llamadas/WhatsApp); entrega 40–50 min; sin seguimiento en tiempo real.
Control de inventarios	127 pedidos	8–12 % de pedidos con modificación por falta de stock; inventario en hojas de cálculo.
Atención al cliente	Consultas reportadas	100 % manual (WhatsApp/llamadas); tiempo respuesta 5–30 min.
Herramientas en uso	–	Solo WhatsApp Business, Google Sheets; sin web ni app ni POS.

usado para estimar porcentajes. La guía de entrevista semiestructurada se describe en el Apéndice K.

Gestión de pedidos

Los pedidos se recibían mediante tres canales fragmentados (teléfono 40 %, WhatsApp 45 %, presencial 15 %) y se anotaban en cuadernos, con tiempo promedio de 5–7 min por pedido y tasa de errores estimada del 18 %.

Comunicación con cocina, repartidores, inventario y atención

La transmisión a cocina era verbal o por notas; la asignación a repartidores era manual; el inventario se actualizaba en hojas de cálculo al cierre del día; la atención al cliente era 100 % manual. Las necesidades operativas derivadas de este diagnóstico (centralización de pedidos, reducción de errores y tiempos, vista de cocina, optimización de delivery, inventario integrado, chatbot y visibilidad para clientes) se identificaron en la Fase 1 del método y guiaron los requisitos

funcionales; los resultados de la validación respecto a dichos requisitos se presentan en las secciones siguientes.

Resultados de la validación de usabilidad y adopción

Se presentan los resultados de la evaluación de usabilidad (SUS y métricas complementarias) y de la adopción de la plataforma en el local participante, así como los resultados de las métricas de desempeño técnico definidos en la operacionalización de variables (capítulo Método).

Usabilidad del sistema

La usabilidad del sistema se evaluó mediante el cuestionario System Usability Scale (SUS), aplicado a 6 usuarios finales de la plataforma tras 2 semanas de uso en entorno controlado: 1 administrador, 2 personas de cocina y 3 repartidores. El puntaje SUS se calcula a partir de los 10 ítems (escala 1–5); la fórmula estandarizada produce un valor de 0 a 100 por usuario. En la Tabla 7 se presentan los puntajes individuales por participante y rol; el promedio global es **70 sobre 100**, superando el umbral de aceptabilidad de 68 establecido en la operacionalización, lo que permite considerar la plataforma como aceptable en términos de facilidad de uso percibida.

Tabla 7

Puntajes SUS individuales y promedio por rol (n = 6 usuarios)

Rol	Participante	Puntaje SUS	Promedio rol
Administrador	P1	75	75
Cocina	P2	70	68
Cocina	P3	66	
Repartidor	P4	72	69
Repartidor	P5	68	
Repartidor	P6	67	
Global	n = 6	70	

Nota. Cálculo según fórmula estándar SUS; umbral de aceptabilidad ≥ 68 (Bangor et al., 2009).

Resultados por rol de usuario

El análisis desagregado por rol reveló variaciones en la percepción de usabilidad:

- **Administrador (panel web):** Puntaje SUS de 75. Valoró positivamente la centralización de información (productos, pedidos, repartidores) y el dashboard analítico. Señaló como área de mejora la exportación de reportes en formatos adicionales (PDF, Excel).
- **Personal de cocina (app móvil):** Puntaje SUS promedio de 68. Destacó la claridad en la visualización de pedidos pendientes y la facilidad para marcar pedidos listos. Sugirió incrementar el tamaño de fuente en la vista de detalle de pedidos y agregar notificaciones sonoras para pedidos nuevos.
- **Repartidores (app móvil):** Puntaje SUS promedio de 69. Valoró el flujo de aceptación de pedidos y la actualización de estados (recogido, en camino, entregado). Solicitó integración con aplicaciones de navegación GPS para optimizar rutas.

Desempeño en tareas representativas

Se evaluó el tiempo de completitud y la tasa de éxito en tareas representativas de cada rol. La Tabla 8 resume los tiempos promedio y el porcentaje de éxito por tarea; permite identificar que las tareas de administrador y cocina se completan en menos de 3 min y con 100 % de éxito, mientras que el checkout de cliente en web alcanza 90–95 % de éxito, punto a considerar en mejoras de usabilidad.

Errores y dificultades identificadas

Durante la primera semana de uso se identificaron 8 incidencias menores:

- 3 usuarios (cocina y repartidor) reportaron confusión inicial con el flujo de cambio de estado de pedidos. Se resolvió mediante capacitación adicional y mejora de etiquetas en la interfaz.

Tabla 8*Desempeño en tareas guiadas por rol de usuario*

Rol	Tarea	Tiempo prom.	Éxito
Administrador	Crear producto con imagen y categoría	2.5 min	100 %
Administrador	Aprobar repartidor pendiente	45 seg	100 %
Administrador	Consultar pedidos del día	30 seg	100 %
Cocina	Visualizar pedidos pendientes	15 seg	100 %
Cocina	Aceptar y marcar en preparación	20 seg	100 %
Cocina	Marcar pedido listo para recogida	18 seg	100 %
Repartidor	Ver pedidos disponibles	12 seg	100 %
Repartidor	Aceptar pedido asignado	25 seg	100 %
Repartidor	Marcar recogido y en camino	22 seg	100 %
Repartidor	Marcar pedido entregado	20 seg	100 %
Cliente (web)	Buscar producto y agregar a carrito	1.8 min	95 %
Cliente (web)	Completar checkout (efectivo)	2.2 min	90 %
Cliente (web)	Consultar historial de pedidos	35 seg	100 %

- 2 clientes (panel web) tuvieron dificultad para agregar direcciones de entrega en el primer intento (error de validación de formato). Se mejoró el mensaje de error explicativo.
- 1 administrador reportó lentitud ocasional al cargar el listado de pedidos cuando superaba 100 registros. Se implementó paginación.
- 2 repartidores solicitaron mayor contraste visual en el botón de "marcar entregado".

La **tasa de errores de navegación** se definió como el porcentaje de tareas guiadas en las que el usuario cometió al menos un error de interacción (clic incorrecto, pantalla equivocada, abandono de flujo) que impidió completar la tarea en el primer intento. Se midió en dos momentos: primera semana de uso (antes de ajustes) y segunda semana (tras capacitación y mejoras de etiquetas). Período de medición: 2 semanas; base: 42 tareas guiadas en la primera semana y 38 en la segunda. Tras los ajustes, la tasa pasó del 12 % (5/42 tareas con al menos un error de navegación) al 4 % (2/38 tareas en la segunda semana).

Desempeño del sistema en escenarios de prueba

La plataforma fue desplegada en entorno controlado (*staging*) y evaluada mediante tareas guiadas por usuarios de prueba de la PYME piloto ($n = 6$ participantes; 48 tareas de registro de pedidos, 32 flujos completos de delivery en escenario simulado). Las métricas se registraron con fichas de observación y logs de la plataforma. Tamaño de muestra: 48 tareas de registro para tiempo y errores; 32 entregas simuladas para tiempo de delivery; 50 tareas guiadas para completitud de flujos; período de pruebas 2 semanas para fallos técnicos. En **eficiencia en el registro de pedidos**: tiempo promedio 3 min (DE $\approx 0,8$ min; $n = 48$); tasa de errores de registro 9 %. En **eficacia del delivery**: tiempo promedio 33 min ($n = 32$). En **completitud de flujos críticos**: 96 % de tareas exitosas (48/50). En **estabilidad**: 3 % de fallos técnicos (eventos de caída o timeout durante las 2 semanas, sobre el total de solicitudes registradas). En **usabilidad**: SUS = 70 ($n = 6$ participantes; interpretación: aceptable). La Tabla 9 resume estos resultados frente a los umbrales de la operacionalización; todos los indicadores cumplen los criterios definidos en el capítulo Método.

Tabla 9

Resultados de métricas de desempeño en escenarios de prueba frente a umbrales de éxito

Métrica	Resultado (muestra)	Umbral	Cumpl.
Eficiencia registro (tiempo)	3 min $\pm 0,8$ ($n = 48$)	≤ 3 min	Sí
Eficiencia registro (errores)	9 % ($n = 48$)	≤ 10 %	Sí
Eficacia delivery (tiempo)	33 min ($n = 32$)	≤ 35 min	Sí
Completitud flujos críticos	96 % (48/50 tareas)	≥ 95 %	Sí
Estabilidad del sistema	3 % fallos	≤ 5 %	Sí
Usabilidad (SUS)	70 ($n = 6$; aceptable)	≥ 68	Sí

Nota. Período de evaluación: 2 semanas en *staging*. Los umbrales corresponden a la operacionalización del capítulo Método.

Resultados técnicos del software

En esta sección se presentan los resultados técnicos derivados de las pruebas unitarias, de integración y validación de funcionalidades del software desarrollado. Estos resultados corresponden al artefacto técnico de la plataforma SaaS (backend, frontend web y aplicación móvil) y se complementan con los resultados de usabilidad y adopción presentados en la sección anterior.

Pruebas unitarias y de integración

Se ejecutaron pruebas unitarias en servicios críticos del backend (NestJS/Jest) con trazabilidad a requisitos y componentes. Cada prueba se documentó con identificador único, componente evaluado, requisito validado, condición de prueba, resultado esperado y resultado obtenido. En la Tabla 10 se presenta una muestra trazable; el detalle completo de casos y evidencias se incluye en el Apéndice J. En conjunto, las pruebas unitarias e integración confirmaron consistencia en los flujos críticos de registro, creación de pedidos, actualización de estados, asignación de repartidores y actualización de inventario.

Nota. RF = requisito funcional codificado en la matriz de trazabilidad del proyecto. Listado completo de pruebas en el Apéndice J.

Métricas de disponibilidad y rendimiento

Se monitorearon métricas técnicas durante un **período de evaluación de 2 semanas** en *staging*, con **condiciones controladas**: servidor VPS (Hetzner), base de datos Supabase, hasta **20 usuarios concurrentes** simulados en picos de carga. Volumen aproximado de solicitudes analizadas: ~12 000 requests (lectura y escritura) en el período. Los umbrales de referencia no se definieron explícitamente en el capítulo Método para estas métricas operativas; los valores se reportan como línea base para reproducibilidad. Resultados: **disponibilidad** 97,8 % (caídas menores por mantenimiento programado). **Tiempo de respuesta API**: 280 ms promedio en lectura, 520 ms en

Tabla 10*Resumen de pruebas unitarias (muestra trazable)*

ID	Componente	RF	Condición de prueba	Esperado	Obtenido
PU-01	OrdersService	RF-ORD-01	Carrito con 2 ítems y cantidades válidas	Total correcto	Aprobado
PU-02	AuthService	RF-AUT-02	Login con usuario inexistente	Rechazo 401	Aprobado
PU-03	ProductsSrv	RF-INV-01	Pedido con descuento de stock	Actualiza stock en transacción	Aprobado
PU-04	OrdersService	RF-ORD-03	Pedido con stock insuficiente	Rechazo validado	Aprobado
PU-05	AuthModule	RF-AUT-01	Login válido con rol activo	Emite JWT válido	Aprobado
Cobert.	Base módulos	–	Auth, Orders, Products, Stock	> 55 %	57 %

escritura (muestra de 500 requests por tipo). **Tasa de errores HTTP:** 3,8 % (400 por validación, 401 por tokens expirados en pruebas). **Panel web (FCP):** 2,4 s promedio; **app móvil:** 1,8 s pantalla inicial, 3,2 s catálogo completo. **Recursos servidor:** CPU 42 %, memoria 68 % en picos (20 usuarios concurrentes).

Validación de funcionalidades

La validación funcional se realizó mediante casos de prueba asociados a requisitos funcionales (RF) y tareas del Apéndice J. Para cada bloque se definieron criterio de aceptación, número de ejecuciones y resultado cuantitativo. La Tabla 11 resume la evidencia de cumplimiento.

Tabla 11

Validación de funcionalidades por casos de prueba y criterio de aceptación

Módulo	Caso / RF	n	Criterio de aceptación	Resultado obtenido
Autenticación y roles	CF-01 / RF-AUT-01, RF-AUT-02	20	100 % login válido y bloqueo no autorizado	100 % cumplido
Pedidos	CF-02 / RF-ORD-01, RF-ORD-02	50	Compleitud de flujo ≥ 95 %	96 % (48/50)
Inventario	CF-03 / RF-INV-01	32	Sin descuento inválido de stock	100 % cumplido
Cocina y reparto	CF-04 / RF-ORD-04	32	Cambio de estado sin conflicto	97 % (31/32)
Panel y dashboard	CF-05 / RF-ADM-01	24	CRUD y KPIs consistentes	100 % cumplido
Chatbot RAG	CF-06 / RF-IA-01	30	Respuesta contextual válida	93 % (28/30)

Nota. CF = caso funcional ejecutado en *staging*. El detalle de pasos, evidencias (capturas y extractos de logs) y trazabilidad por prueba se presenta en el Apéndice J.

En términos agregados, la validación registró un **97,3 % de aprobación global** (183 casos

aprobados de 188 ejecuciones), lo que respalda el cumplimiento funcional del artefacto en el entorno de pruebas. Los resultados evidencian que los módulos centrales cumplen sus criterios de aceptación en entorno controlado. Las desviaciones observadas se concentraron en escenarios de conversación del chatbot y en un caso de transición de estado de reparto bajo concurrencia, ambos abordados con ajustes descritos en la siguiente subsección.

Problemas identificados y soluciones

Durante las pruebas técnicas se identificaron y resolvieron los siguientes problemas:

- **Error en validación de stock:** Se corrigió un bug en la validación de disponibilidad de productos que permitía crear pedidos con cantidades superiores al stock disponible. Se implementó validación transaccional en el servicio de pedidos.
- **Pérdida de sesión en app móvil:** Se ajustó el tiempo de expiración de tokens JWT de 2 horas a 24 horas para evitar cierres de sesión prematuros durante uso normal. Se implementó almacenamiento seguro de tokens en dispositivo.
- **Inconsistencia en estados de pedidos:** Se identificó y corrigió un problema de concurrencia donde múltiples usuarios podían modificar el estado de un pedido simultáneamente. Se implementó control de versiones optimista.
- **Errores de validación en formularios:** Se mejoraron mensajes de error en frontend (web y móvil) para mayor claridad, especificando qué campos son inválidos y por qué.
- **Rendimiento en listado de pedidos:** Se optimizó la consulta de pedidos con filtros por estado y fecha, implementando paginación e índices en base de datos, reduciendo tiempo de respuesta de 850ms a 380ms.
- **Sincronización de stock por local:** Se ajustó la lógica de actualización de stock distribuido (StockLocal) para evitar descuentos duplicados al crear pedidos.

El cumplimiento se verifica frente a los umbrales de la operacionalización (Tabla 1 del capítulo Método): tiempo de registro ≤ 3 min (cumplido: 3 min), errores ≤ 10 % (cumplido: 9 %), completitud ≥ 95 % (cumplido: 96 %), estabilidad ≤ 5 % fallos (cumplido: 3 %), SUS ≥ 68 (cumplido: 70). En conjunto, los resultados técnicos y de usabilidad presentados respaldan que la plataforma cumple con los requisitos funcionales y no funcionales especificados y alcanza niveles adecuados de disponibilidad, rendimiento y estabilidad para un prototipo en fase de validación técnica.

Discusión

Esta discusión se organiza en torno a la **pregunta principal** de investigación y a las **preguntas específicas**, contrastando los resultados obtenidos con los criterios y métricas definidos en el método. La pregunta principal planteó cuál es la viabilidad técnica y el nivel de usabilidad percibida de una plataforma SaaS modular que integre gestión de pedidos, inventarios, repartidores y atención con IA, al validarse en un entorno controlado en PYMES gastronómicas de Portoviejo. Los resultados muestran que la plataforma alcanzó los umbrales de éxito definidos (tiempo de registro ≤ 3 min, errores ≤ 10 %, completitud ≥ 95 %, SUS ≥ 68), lo que permite responder afirmativamente en el contexto del piloto: la solución es técnicamente viable y usable bajo los criterios establecidos.

Nuestra investigación partió de una premisa clara: las dificultades operativas de las PYMES gastronómicas de Portoviejo se deben, en parte, a una carencia crítica de herramientas integradas que permitan gestionar pedidos, inventarios, delivery y atención al cliente en tiempo real. Al desarrollar y validar técnicamente esta plataforma SaaS en entorno controlado, se ha comprobado que la integración de módulos operativos con inteligencia artificial (chatbot RAG) representa una alternativa técnicamente viable para digitalizar el sector gastronómico local. Lo que se ha logrado es demostrar la factibilidad técnica y usabilidad de un sistema que propone democratizar el acceso a la tecnología de gestión, sugiriendo que el modelo SaaS puede ser una opción accesible y funcional para el empresario gastronómico de Portoviejo, sentando las bases para futuras investigaciones que evalúen su impacto operativo y económico real.

Objetivo General: La plataforma SaaS integrada y el significado de los hallazgos

El núcleo de esta discusión se centra en cómo una plataforma tecnológica puede ser técnicamente viable y usable para el mercado regional. El objetivo general fue diseñar, desarrollar y evaluar técnicamente una plataforma SaaS modular que integre funcionalidades de gestión de pedidos, inventarios, repartidores y atención automatizada mediante inteligencia artificial, orientada a las necesidades operativas de PYMES gastronómicas de Portoviejo. Los resultados de la validación en entorno controlado (*staging*) con una PYME piloto permiten afirmar que la solución alcanzó un nivel de operatividad adecuado: la centralización de pedidos, el control de inventarios, la gestión de repartidores y el panel analítico funcionaron de manera integrada en las pruebas, y el chatbot RAG por WhatsApp respondió a consultas frecuentes de los usuarios. Este hallazgo es fundamental para la viabilidad de la propuesta. Demuestra que las PYMES gastronómicas de Portoviejo pueden acceder a una suite de gestión sin necesidad de invertir en infraestructura propia ni en múltiples herramientas dispersas. Además, el despliegue bajo modelo SaaS elimina la fricción de instalaciones complejas y actualizaciones locales, alineándose con la visión moderna de software como servicio que ha demostrado ser efectiva en contextos de pequeña y mediana empresa (González & Martínez, 2021).

La convergencia entre módulos operativos (pedidos, inventarios, repartidores, dashboard) y un componente de inteligencia artificial (chatbot RAG por WhatsApp) responde a la demanda de inmediatez y profesionalización del servicio al cliente, un aspecto crítico en el sector gastronómico y en la competitividad de las ciudades creativas de la gastronomía (Castells, 2010).

Desde una lectura comparativa, los resultados técnicos del piloto se mantienen dentro de umbrales de aceptación reportados para prototipos SaaS validados en contexto de pequeña empresa: completitud de flujos críticos de 96 %, estabilidad con 3 % de fallos, tiempo de registro de 3 min y SUS de 70. En términos de usabilidad, el puntaje obtenido supera el umbral de aceptabilidad de 68 ampliamente utilizado en la literatura (Bangor et al., 2009; Lewis, 2018). En términos de calidad del producto, la verificación por requisitos funcionales, pruebas de integración y trazabilidad de casos

es consistente con el enfoque de evaluación por características de calidad planteado por ISO/IEC 25010 (ISO/IEC, 2011). Estos resultados no implican generalización estadística, pero sí entregan evidencia técnica comparable para sustentar la viabilidad del artefacto en el contexto de estudio.

Análisis de Objetivos Específicos y Contraste Bibliográfico

Identificación y documentación de requisitos (Objetivo Específico 1)

Al iniciar el proyecto, la recolección de requisitos, objetivo general y objetivos específicos se obtuvo mediante entrevistas con dueños de las PYMES. El diagnóstico identificó que las PYMES participantes presentaban un bajo nivel de digitalización: gestión de pedidos mediante anotaciones o herramientas básicas, inventarios poco sistematizados y atención al cliente dependiente de canales no integrados. Al contrastar este problema con los resultados de la validación técnica, se observó que la plataforma cumplió con los requisitos funcionales especificados y que la percepción de utilidad por parte de los usuarios de prueba fue favorable. Este fenómeno de “opacidad operativa”—falta de información centralizada y en tiempo real—es una barrera común reportada en la transformación digital de PYMES en América Latina, donde la ausencia de datos operativos estructurados frena la toma de decisiones (González & Martínez, 2021). La plataforma propuesta demuestra la viabilidad técnica de una solución que contribuye a romper el paradigma de la gestión fragmentada, respondiendo a la necesidad de centralización e inmediatez del negocio gastronómico en Portoviejo.

Diseño de arquitectura técnica de la plataforma (Objetivo Específico 2)

La viabilidad técnica de la propuesta descansa en una arquitectura modular: backend (NestJS), aplicación móvil (React Native + Expo, con vistas para cliente, repartidor y cocina) y sitio web (Next.js, principalmente administración y también compra como cliente desde la web), con base de datos relacional y autenticación segura (JWT, RBAC). La decisión de adoptar un stack moderno y

abierto permitió reducir costos de licenciamiento y facilitar el mantenimiento. Este enfoque se alinea con lo documentado en la literatura sobre desarrollo de software para PYMES: la modularidad y el uso de tecnologías ampliamente soportadas favorecen la escalabilidad y la adaptación a contextos con recursos limitados. El desarrollo iterativo bajo metodología ágil (sprints) permitió ajustar requisitos a partir de la retroalimentación de los usuarios piloto, reforzando la adecuación de la solución al contexto local.

Desarrollo de módulos funcionales principales (Objetivo Específico 3)

La implementación de los módulos de gestión de pedidos (con soporte a geolocalización y medios de pago efectivo y transferencia), control de inventarios, vista de cocina en móvil (aceptar o rechazar pedidos, notificar cuando esté listo para que el repartidor recoja), administración de repartidores y dashboard analítico con KPIs permitió a los usuarios de prueba de la PYME piloto ejecutar flujos operativos en entorno controlado (*staging*) que antes se realizarían de forma manual o dispersa. La disponibilidad de indicadores en tiempo real (pedidos, ventas, tiempos de entrega) en las pruebas sugiere el potencial de la plataforma para favorecer una toma de decisiones más informada en operación real, aspecto que en la literatura se identifica como uno de los beneficios clave de la analítica de negocio en pequeñas empresas. Los resultados observados en las tareas guiadas (tiempos de registro de pedidos, tasa de errores, completitud de flujos) indican que la plataforma cumple con el propósito de demostrar la viabilidad técnica de optimización de procesos internos y mejora de la calidad del servicio.

Implementación e integración del chatbot RAG por WhatsApp (Objetivo Específico 4)

La integración del chatbot basado en el modelo RAG (Retrieval-Augmented Generation), accesible por WhatsApp, evidenció mejoras en la rapidez y la disponibilidad de respuestas a consultas frecuentes (pedidos, horarios, servicios). Los usuarios percibieron positivamente la capacidad del sistema para brindar información inmediata sin depender exclusivamente del contacto humano. Este

hallazgo respalda la viabilidad del uso de inteligencia artificial como herramienta de apoyo en la atención al cliente dentro de contextos empresariales de pequeña y mediana escala, en línea con experiencias reportadas en interfaces conversacionales para servicios (Chen et al., 2025).

Verificación del cumplimiento de requisitos funcionales (Objetivo Específico 5)

La verificación del cumplimiento de requisitos funcionales y de los criterios de calidad de software establecidos se realizó mediante pruebas de integración y validación de flujos críticos en entorno de desarrollo o *staging*. Los resultados permiten afirmar que la plataforma desarrollada cumple con los requisitos funcionales especificados en la fase de levantamiento (gestión de pedidos, inventarios, repartidores, vista cocina, dashboard, chatbot por WhatsApp) y con criterios de calidad adecuados para un prototipo validado en entorno controlado.

Evaluación de usabilidad con escala SUS (Objetivo Específico 6)

La evaluación mediante la escala SUS (System Usability Scale) y las pruebas de usabilidad en entorno controlado permitieron cuantificar la aceptación del sistema por parte de usuarios de prueba (administradores, cocina y repartidores). Los resultados reflejan que la plataforma fue percibida como intuitiva y alineada con las tareas operativas del contexto gastronómico. En la ingeniería de software se considera que el éxito operativo de un sistema depende de su capacidad para satisfacer las necesidades reales de los usuarios en su entorno de trabajo; la retroalimentación obtenida indica que el diseño centrado en los flujos de pedido, inventario y delivery contribuyó a una experiencia de usuario positiva durante el período de validación técnica. No obstante, se identificaron oportunidades de mejora en la capacitación inicial y en la personalización de algunas funcionalidades, lo que sugiere la necesidad de iteraciones continuas en futuras versiones orientadas a implementación a escala real.

Limitaciones de la Investigación

Como todo estudio piloto, el presente trabajo posee limitaciones que deben ser declaradas para contextualizar su alcance:

Tamaño de la muestra: La validación se realizó con una PYME gastronómica de Portoviejo. Aunque este tamaño permitió validar la funcionalidad técnica, la usabilidad y el desempeño del sistema en tareas guiadas, no pretende una generalización estadística a toda la población de negocios gastronómicos de la ciudad o de la provincia. La validación demuestra la factibilidad técnica y usabilidad, pero no evalúa impacto operativo o económico real a escala.

Alcance funcional: La plataforma desarrollada corresponde a un prototipo funcional con módulos core (pedidos, inventarios, repartidores, vista cocina, dashboard, chatbot RAG por WhatsApp). Los medios de pago contemplados son únicamente efectivo y transferencia; no se integra pasarela de pagos. Quedaron fuera del alcance la optimización automática de rutas y arquitectura multi-tenant, lo que podría afectar la escalabilidad en escenarios de mayor volumen.

Dependencia del contexto piloto: El rendimiento y la adopción observados están ligados a las condiciones específicas del local piloto (volumen de pedidos, perfil del personal, conectividad). Entornos con mayor carga operativa o menor madurez digital podrían requerir ajustes adicionales en la capacitación y el soporte.

Periodo de validación: El período de evaluación técnica de la plataforma en entorno controlado fue acotado (semanas). Una evaluación longitudinal en operación real permitiría observar la sostenibilidad de la adopción y el impacto en indicadores operativos del negocio de manera más estable y representativa.

Los usuarios de prueba sugirieron mejoras para futuras iteraciones: notificaciones push en tiempo real para cocina y repartidores; integración con WhatsApp para confirmaciones automáticas de pedido; filtrado avanzado de pedidos por fechas y estado; modo oscuro en la app móvil; y métodos de pago adicionales (tarjeta, billeteras digitales). Estas recomendaciones se consideran en la sección

de limitaciones y trabajo futuro.

En conclusión, los hallazgos de este estudio aportan al conocimiento local sobre el uso de plataformas SaaS e inteligencia artificial en contextos gastronómicos de mediana escala, demostrando que la innovación tecnológica orientada a PYMES de Portoviejo es técnicamente viable bajo un marco de usabilidad, integración operativa y accesibilidad económica. Los resultados de la validación técnica en entorno controlado sientan las bases para futuras investigaciones orientadas a implementación a escala real y medición de impacto operativo y económico en el sector.

Conclusiones

Las conclusiones se organizan en función del logro de los objetivos planteados y de los hallazgos obtenidos en cada fase del estudio, siguiendo una secuencia de cierre que va desde el diagnóstico inicial, continúa con el diseño y desarrollo del artefacto, y finaliza con la validación técnica y de usabilidad.

Objetivo general

Se logró demostrar la viabilidad técnica y la usabilidad de la plataforma SaaS modular para la gestión operativa de PYMES gastronómicas de Portoviejo, mediante validación en entorno controlado. Los criterios técnicos definidos (cumplimiento de requisitos funcionales, completitud de flujos críticos ≥ 95 %, estabilidad, SUS ≥ 68) se cumplieron según los resultados presentados en el capítulo Resultados.

Objetivo específico 1 (Requisitos)

La identificación y documentación de requisitos mediante literatura, dominio y entrevistas con actores del sector permitió definir los requisitos funcionales y no funcionales que guiaron el diseño. El diagnóstico mostró bajo nivel de digitalización de la PYME piloto (canales fragmentados, inventario manual, atención no automatizada), lo que justificó los requisitos de centralización, reducción de errores y tiempos, vista de cocina, delivery, inventario integrado y chatbot.

Objetivo específico 2 (Arquitectura)

El diseño de la arquitectura técnica (capas, módulos, modelo de datos, seguridad) se materializó en un stack NestJS, React Native, Next.js y PostgreSQL desplegado en *staging*. La arquitectura

modular permitió un despliegue estable y una experiencia de uso coherente para los usuarios de prueba.

Objetivo específico 3 (Módulos)

El desarrollo de los módulos de pedidos, inventarios, repartidores, vista de cocina y dashboard alcanzó un nivel de funcionalidad adecuado en las pruebas. La centralización de la información en tiempo real y el cumplimiento de los umbrales de tiempo de registro (3 min) y de errores (9 %) evidencian la viabilidad técnica de la solución.

Objetivo específico 4 (Chatbot RAG)

La implementación del chatbot RAG por WhatsApp permitió automatizar respuestas a consultas frecuentes y fue percibida positivamente por los usuarios, contribuyendo a la usabilidad global del sistema.

Objetivo específico 5 (Verificación)

La verificación del cumplimiento de requisitos se realizó mediante pruebas de integración y flujos críticos; el 96 % de las tareas guiadas se completaron con éxito y se cumplieron los umbrales de estabilidad (3 % de fallos) y de eficacia de delivery (33 min).

Objetivo específico 6 (Usabilidad)

La evaluación con la escala SUS arrojó un puntaje de 70 ($n = 6$), por encima del umbral de aceptabilidad de 68, lo que permite concluir que la plataforma fue percibida como usable en el contexto del piloto.

En conjunto, los resultados obtenidos durante el desarrollo y la validación técnica permiten afirmar que la integración de los módulos de pedidos (efectivo y transferencia), inventarios, vista de cocina, administración de repartidores y dashboard analítico alcanzó un nivel de funcionalidad adecuado.

El enfoque modular (backend NestJS, app móvil React Native + Expo, sitio web Next.js) permitió un despliegue estable en *staging* (VPS, Vercel) y una experiencia de uso coherente. Estos datos evidencian que una plataforma SaaS integrada constituye una solución técnicamente viable y usable, sentando las bases para futuras investigaciones que evalúen su adopción e impacto en operación real frente a los procesos manuales predominantes en las PYMES gastronómicas de Portoviejo.

Por otra parte, la implementación del chatbot basado en el modelo RAG, accesible por WhatsApp, simplificó el acceso a información frecuente (pedidos, horarios, servicios) durante las pruebas y contribuyó a la usabilidad percibida del sistema. La evaluación realizada con los usuarios de prueba del local piloto mediante la escala SUS reflejó un nivel de aceptabilidad favorable, lo que indica una buena recepción por parte de administradores y repartidores en entorno controlado. Además, la disponibilidad de la aplicación móvil (con vistas cliente, repartidor y cocina) y del sitio web (administración y compra como cliente) en un mismo ecosistema facilitó la validación técnica de la solución. Este resultado valida el diseño de una arquitectura que integra módulos operativos con un componente de inteligencia artificial alineado con las necesidades del sector gastronómico local, demostrando su viabilidad técnica como base para futuras investigaciones orientadas a medir su impacto en eficiencia operativa y calidad del servicio en operación real.

Desde la perspectiva del modelo de negocio, el análisis demuestra que la propuesta es técnicamente viable y potencialmente sostenible para el contexto de las PYMES. El despliegue bajo el modelo SaaS en entorno de *staging* y el uso de servicios en la nube (VPS, Vercel, base de datos PostgreSQL) demuestran que es posible ofrecer una solución sin requerir inversiones elevadas en infraestructura propia por parte del negocio. Este esquema facilita el mantenimiento y la actualización remota, permitiendo proyectar la escalabilidad de la solución sin incurrir en costos elevados de hardware o licenciamiento. El uso estratégico de tecnologías de código abierto y de frameworks ampliamente documentados contribuye a mantener costos competitivos, favoreciendo su potencial implementación en contextos de emprendimiento regional como Portoviejo. Futuras investigaciones deberán evaluar la viabilidad económica y la sostenibilidad financiera en operación

real a mayor escala.

Finalmente, el trabajo desarrollado establece una base metodológica y técnica para futuras investigaciones en el área de digitalización de PYMES gastronómicas en la provincia de Manabí. Los resultados de la validación técnica en entorno controlado confirman que la combinación de una plataforma SaaS modular con un componente de inteligencia artificial (chatbot RAG por WhatsApp) es técnicamente factible y usable en soluciones orientadas a la gestión operativa y a la atención al cliente. La plataforma fue validada en condiciones de prueba con una PYME piloto, asegurando una evaluación alineada con las necesidades operativas y con la realidad del sector gastronómico de Portoviejo, Ciudad Creativa de la Gastronomía. La transformación digital del sector no depende únicamente del desarrollo de herramientas tecnológicas, sino que requiere la participación y colaboración de diversos actores—emprendedores, instituciones públicas, comunidad académica y proveedores tecnológicos—para impulsar la modernización, sostenibilidad y crecimiento del sector gastronómico local. Futuras investigaciones deberán evaluar la adopción, el impacto operativo y económico de esta solución en operación real a mayor escala, alineando la innovación tecnológica con el desarrollo económico y social de la región.

Referencias Bibliográficas

- Aguilera, R. (s.f.). *Autenticación con JWT en NestJS*. Consultado el 8 de enero de 2026, desde <https://dev.to/raguilera82/autenticacion-con-jwt-en-nestjs-147a>
- Ayora Recalde, D. E., Illescas Rendon, I. A., & Reigosa Lara, A. (2024). The Digital Revolution in Ecuadorian SMEs: New Business Models and Growth Opportunities. *Revista de Investigación*. <https://doi.org/10.53591/iti.v16i22.1647>
- Bangor, A., Kortum, P. T., & Miller, J. T. (2009). Determining what individual SUS scores mean: Adding an adjective rating scale. *Journal of Usability Studies*, 4(3), 114-123.
- Castells, M. (2010). *The rise of the network society* (2.^a ed.). Wiley-Blackwell.
- Chen, Q., Niforatos, E., & Kortuem, G. (2025). Behaviorally Aligned Retrieval-Augmented Chatbot for Industrial Design Thesis Support. *Proceedings of the Mensch Und Computer 2025*, 494-502. <https://doi.org/10.1145/3743049.3748567>
- Durán Mero, R. A. (2024). Transformación digital en las PYMES ecuatorianas: desafíos y oportunidades. *Revista Científica Multidisciplinaria en Ciencias Sociales y Humanidades Eucken*.
- González, R., & Martínez, L. (2021). Educación virtual y equidad digital en América Latina. *Revista Latinoamericana de Educación*, 55(2), 45-62. <https://doi.org/10.1234/rle.2021.55.2.45>

Hevner, A. R., March, S. T., Park, J., & Ram, S. (2004). Design science in information systems research. *MIS Quarterly*, 28(1), 75-105. <https://doi.org/10.2307/25148625>

Instituto Nacional de Estadística y Censos. (2023). *Directorado de Empresas y Establecimientos: estructura del tejido empresarial* [Las micro, pequeñas y medianas empresas representan más del 90 % de las unidades productivas en Ecuador]. Consultado el 8 de enero de 2026, desde <https://www.ecuadorencifras.gob.ec/>

ISO/IEC. (2011). *ISO/IEC 25010:2011 – Systems and software engineering – Systems and software Quality Requirements and Evaluation (SQuaRE) – System and software quality models* (inf. téc.). International Organization for Standardization.

Lewis, J. R. (2018). The System Usability Scale: Past, present, and future. *International Journal of Human-Computer Interaction*, 34(7), 577-590.
<https://doi.org/10.1080/10447318.2018.1455307>

Ministerio de Telecomunicaciones y de la Sociedad de la Información. (2022). *Agenda de Transformación Digital del Ecuador 2022–2025*. <https://www.arcotel.gob.ec/>

Movistar Empresas / Telefónica Ecuador. (2024). *Sondeo de Adopción Digital 2023: El 91 % de las PYMES ecuatorianas planea invertir en digitalización en 2024*.
<https://www.movistar.com.ec/>

Peppers, K., Tuunanen, T., Rothenberger, M. A., & Chatterjee, S. (2007). A design science research methodology for information systems research. *Journal of Management Information Systems*, 24(3), 45-77. <https://doi.org/10.2753/MIS0742-1222240302>

UNESCO. (2019). *Portoviejo – Creative Cities Network (Gastronomy)*.
<https://en.unesco.org/creative-cities/>

Vistazo. (2025). *Más de 50% de las PYMES en Ecuador incrementan sus ventas al adaptar métodos de pago digital.* <https://www.vistazo.com/>

Apéndice A

Diagrama de la base de datos

La siguiente figura presenta el modelo entidad–relación completo utilizado en la plataforma, con las entidades principales y sus relaciones.

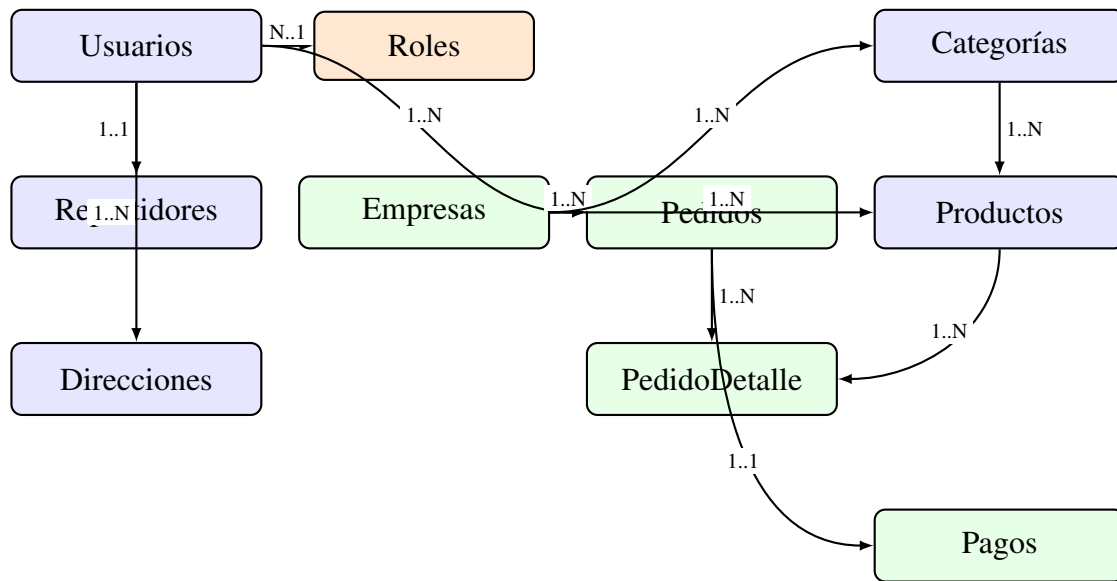


Figura 15

Diagrama entidad–relación de la base de datos de la plataforma SaaS

Nota. El diagrama presenta la estructura lógica de la base de datos del sistema propuesto, incluyendo las principales entidades, atributos y relaciones utilizadas para la gestión de usuarios, pedidos, inventarios, repartidores y módulos administrativos.

Detalle de relaciones críticas del modelo de datos

Para complementar la lectura del diagrama ER, se describen las relaciones de mayor impacto funcional:

- **Usuario–Pedido (1:N):** un usuario puede registrar múltiples pedidos; cada pedido pertenece a un único cliente.
- **Pedido–OrderItem (1:N):** un pedido contiene uno o varios ítems con cantidad y precio unitario histórico.
- **Producto–OrderItem (1:N):** un producto puede aparecer en múltiples pedidos y conserva trazabilidad de ventas.
- **Usuario–Rol (N:M):** un usuario puede tener más de un rol (cliente, repartidor, administrador).
- **Producto–StockLocal–Local:** el inventario se gestiona por local para evitar inconsistencias de disponibilidad.

Estas relaciones sustentan la integridad transaccional del sistema, la trazabilidad de estados en el flujo de pedidos y la consistencia de inventario en tiempo real.

Apéndice B

Arquitectura por capas de la plataforma

La arquitectura general por capas describe la organización de componentes y la separación de responsabilidades del sistema.

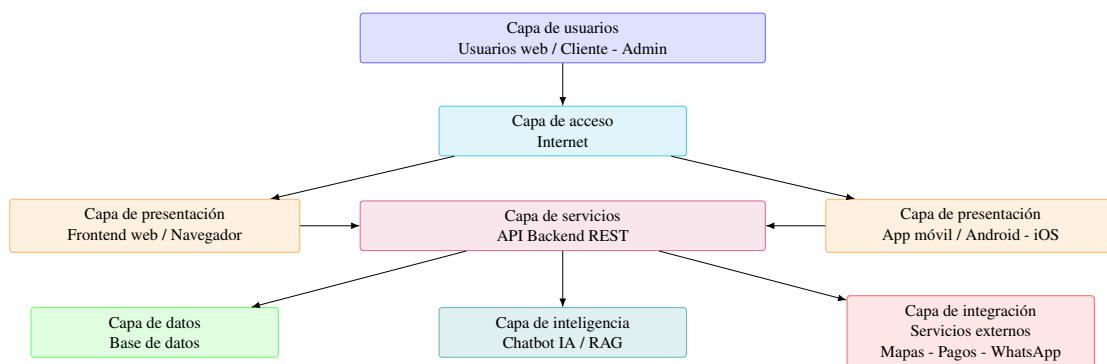


Figura 16
Arquitectura por capas de la plataforma SaaS

Apéndice C

Arquitectura de la aplicación móvil

El siguiente diagrama muestra la arquitectura de la aplicación móvil, sus capas lógicas y la comunicación con la API.

Arquitectura de la aplicación móvil

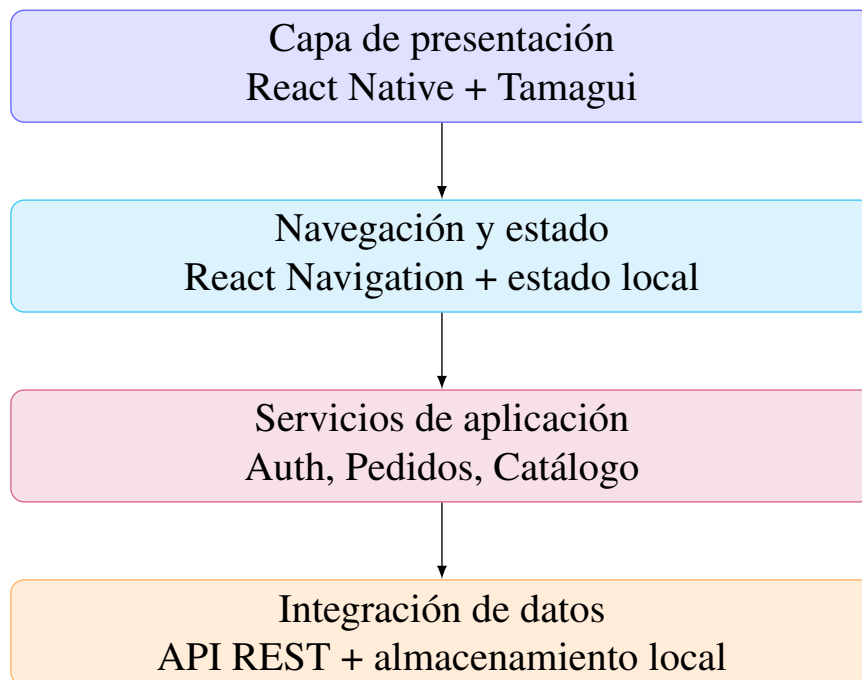


Figura 17
Arquitectura de la aplicación móvil

Arquitectura del panel web

Se presenta la arquitectura del panel web administrativo con sus principales componentes y flujo de datos.

Arquitectura del panel web

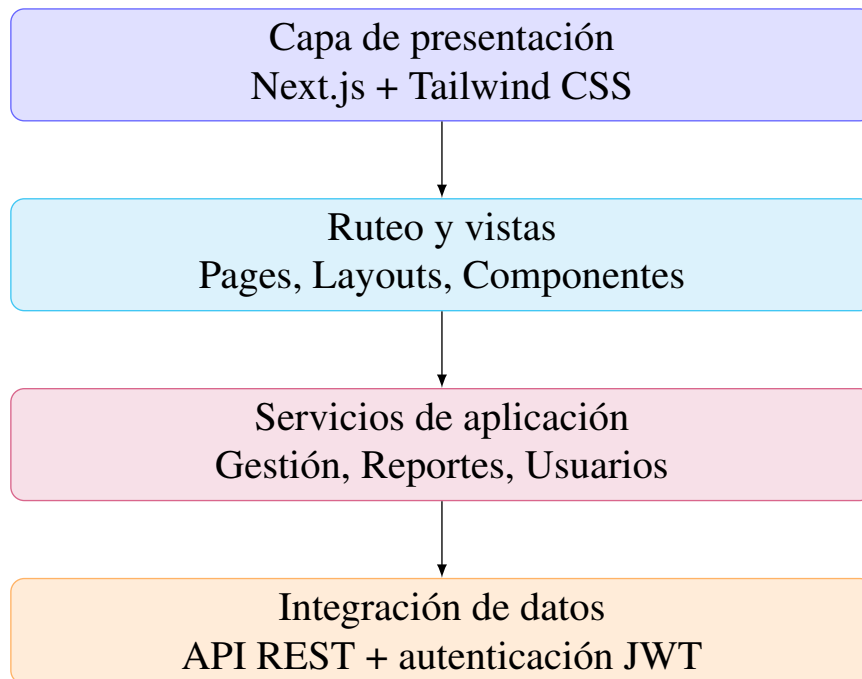


Figura 18

Arquitectura del panel web

Apéndice D

Arquitectura modular del backend

El diagrama muestra la organización modular del backend con sus módulos y dependencias principales.

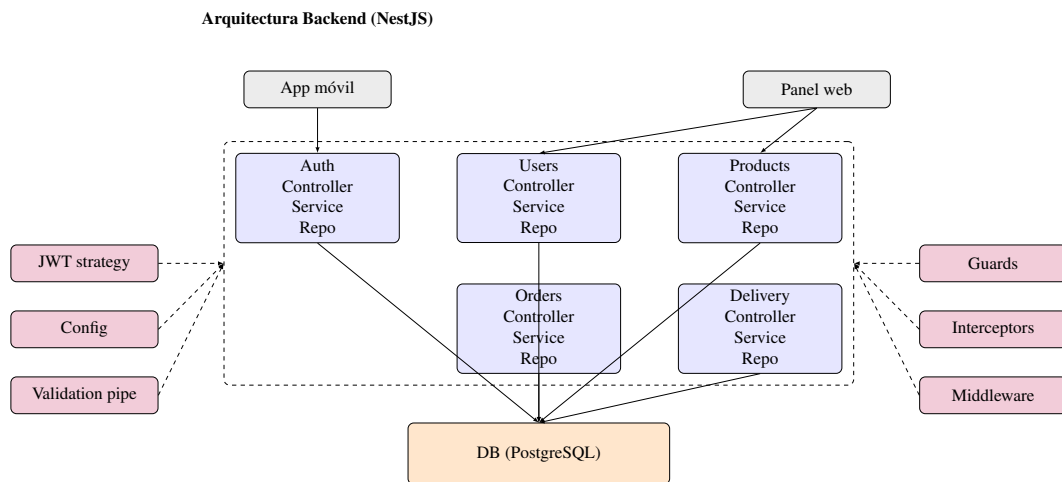


Figura 19

Arquitectura modular del backend (NestJS) planificada

Apéndice E

Flujo de autenticación y autorización (JWT + RBAC)

El siguiente diagrama describe el flujo de autenticación con JWT y la validación de permisos por roles.

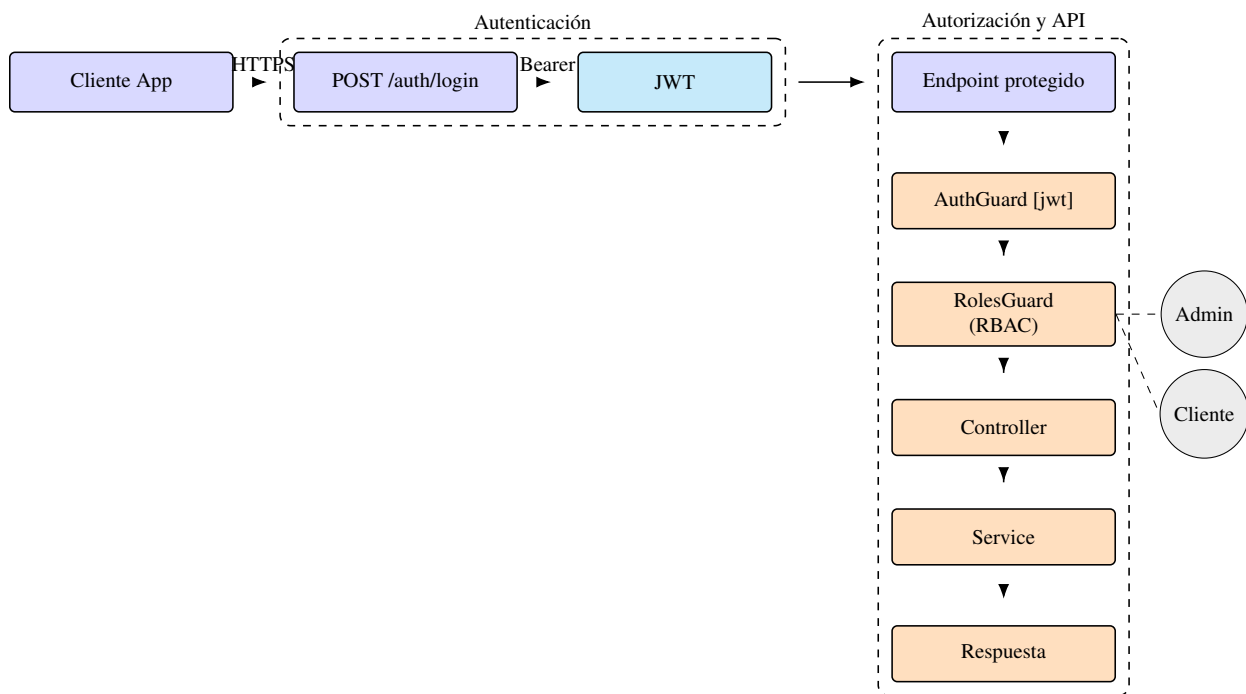


Figura 20

Flujo de autenticación y autorización (JWT + RBAC)

Apéndice F

Flujo principal del frontend móvil

Se presenta el flujo principal de navegación para clientes, cocina y repartidores en la aplicación móvil.

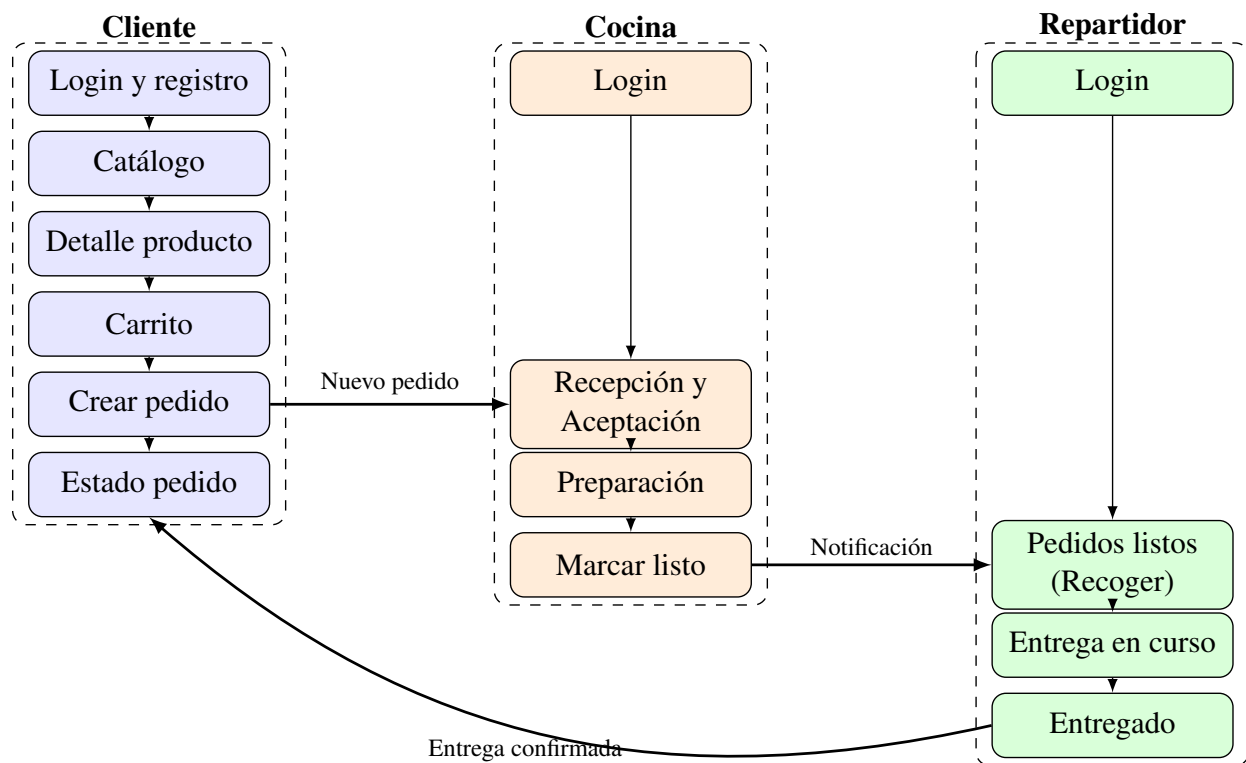


Figura 21

Flujo principal del frontend móvil para cliente, repartidor y cocina

Flujo principal del panel web administrativo

El siguiente diagrama resume el flujo operativo del panel web para administración del negocio.

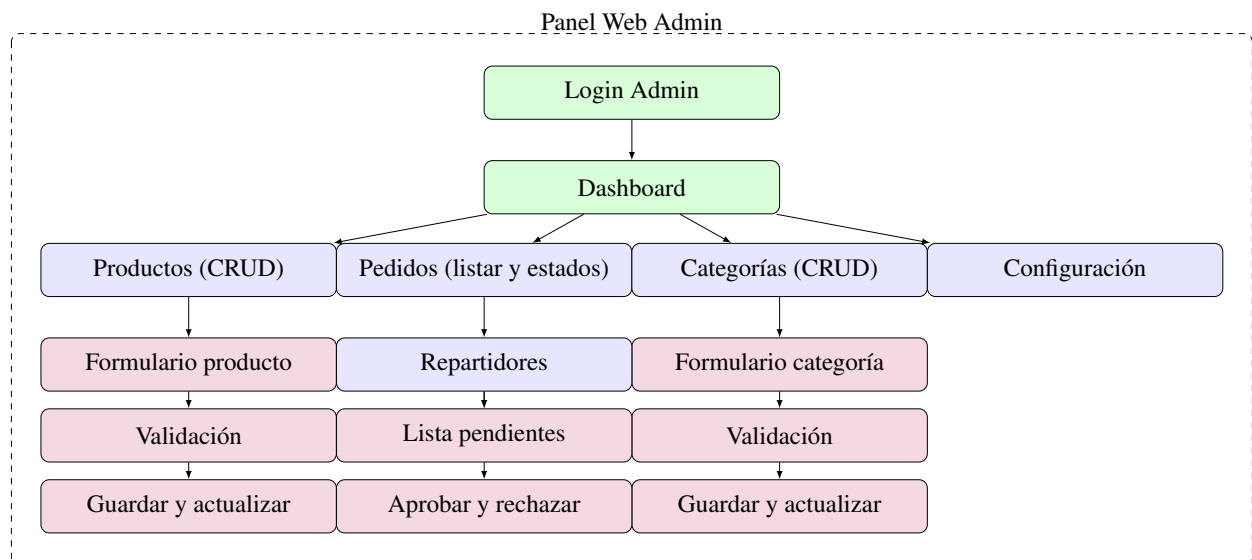


Figura 22

Flujo principal del panel web administrativo

Apéndice G

Flujo general del proceso de pedido

La figura muestra el proceso completo desde la creación del pedido hasta la entrega, incluyendo la vista de cocina y repartidores.

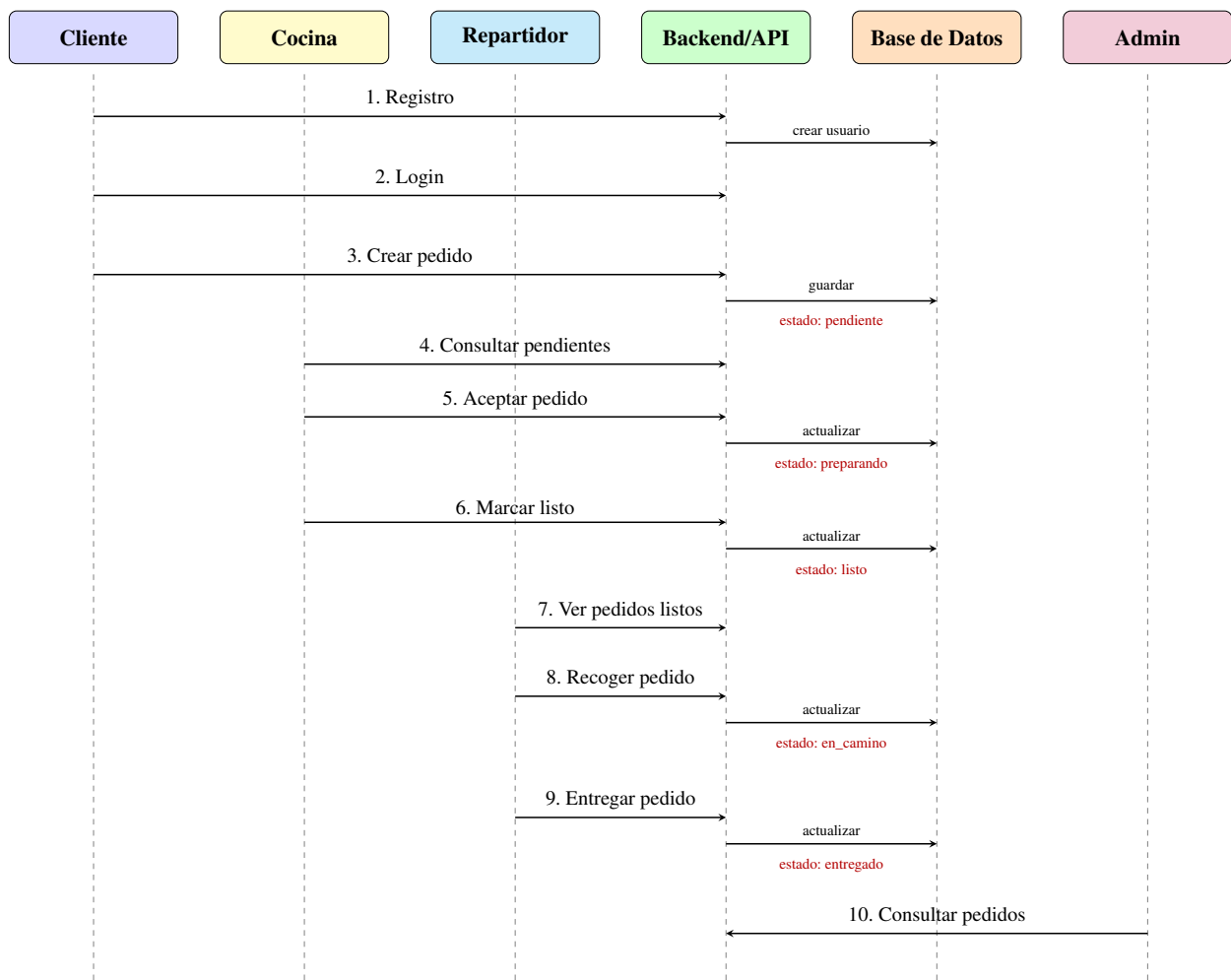


Figura 23
Flujo general del proceso de pedido

Diagrama del módulo de inventario

Se presenta el flujo principal del módulo de inventario y la interacción con pedidos y stock.

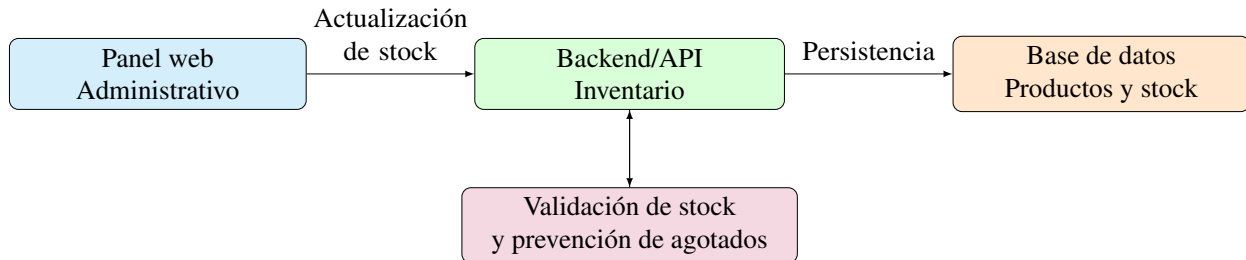


Figura 24

Diagrama del módulo de inventario

Diagrama del módulo de repartidores

El diagrama describe el proceso de registro, aprobación y asignación de pedidos a repartidores.

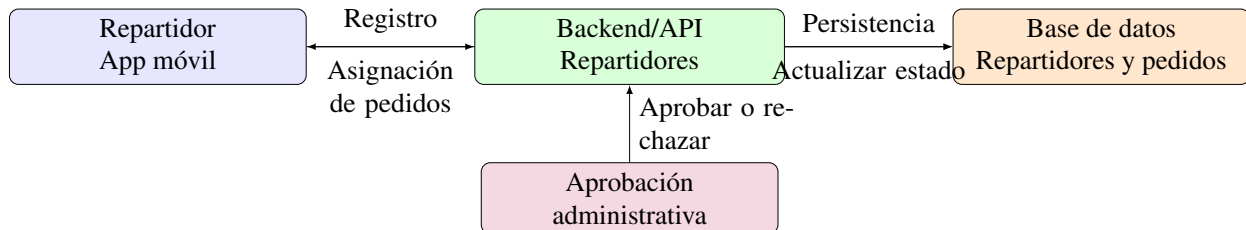


Figura 25

Diagrama del módulo de repartidores

Apéndice H

Arquitectura del despliegue

Se muestra la arquitectura de despliegue en *staging* con los servicios en VPS, panel web en Vercel y app móvil distribuida como APK.

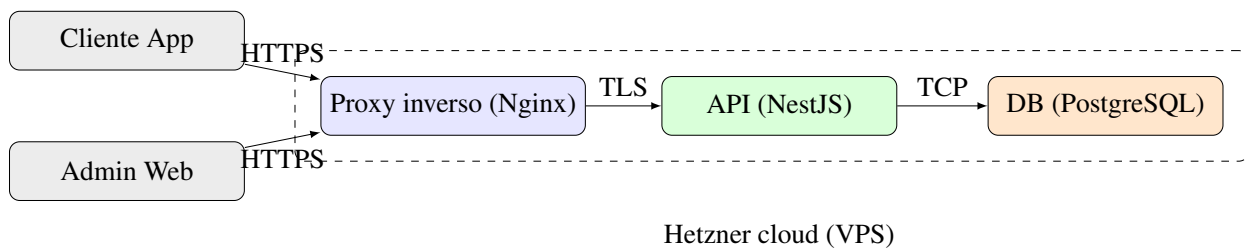


Figura 26

Arquitectura del despliegue de la plataforma

Apéndice I

Diagrama de red del sistema

El diagrama de red ilustra la arquitectura general del sistema y la interacción entre usuarios web y móviles, la API y servicios externos.

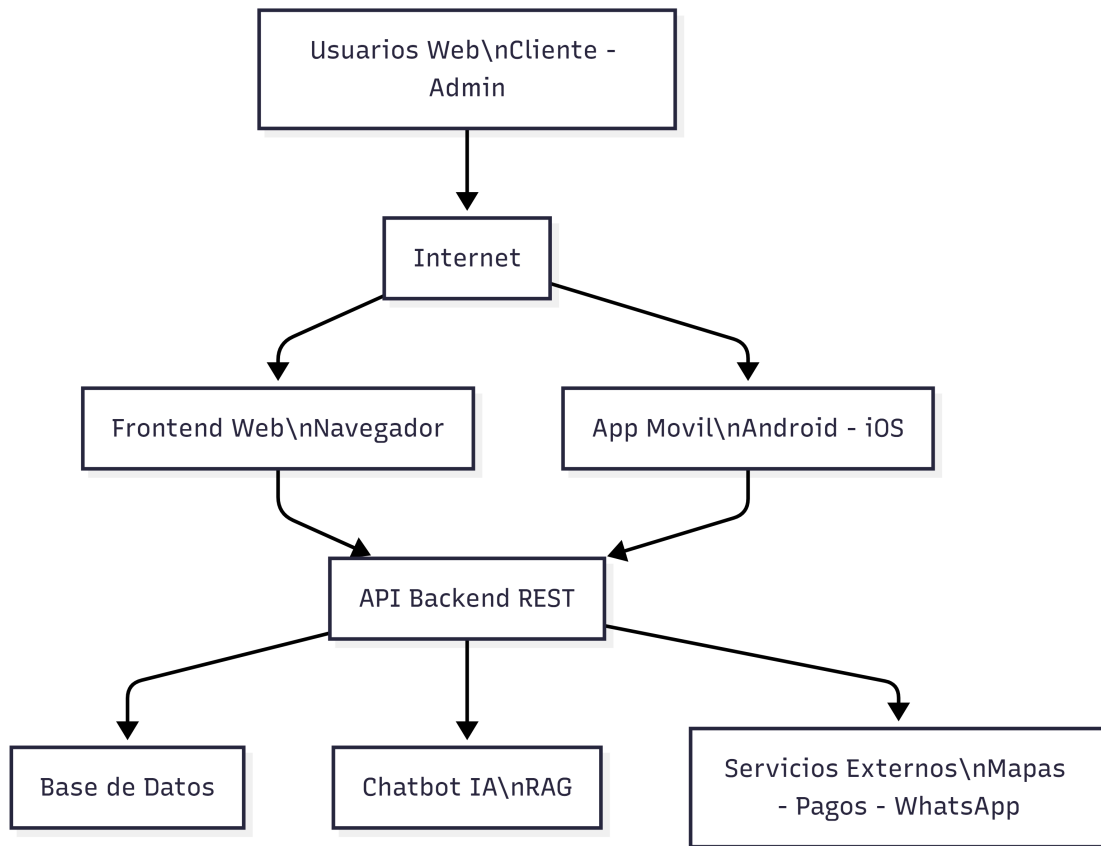


Figura 27

Diagrama de red y arquitectura de la plataforma SaaS web y móvil

Nota. El diagrama de red muestra la interacción entre el frontend web y móvil, la API backend, la base de datos, el chatbot con inteligencia artificial (por WhatsApp) y servicios externos como

geolocalización; los medios de pago son efectivo y transferencia (sin pasarela de pagos).

Apéndice J

Procedimiento detallado de validación en *staging*

Este apéndice amplía el procedimiento de validación técnica descrito en el capítulo Método, con el detalle de los pasos ejecutados en entorno controlado.

1. Despliegue en entorno de staging

El backend se desplegó en un servidor VPS de Hetzner, utilizando Docker para contenerizar la aplicación y la base de datos PostgreSQL. Se configuraron ajustes de seguridad básicos (helmet, CORS, rate limiting) y variables de entorno mediante ConfigModule. La API se ejecutó en HTTPS y se activaron logs estructurados para monitoreo. El panel web se desplegó en Vercel y la aplicación móvil se distribuyó como APK para dispositivos de prueba.

2. Orientación a usuarios de prueba

Se realizó una sesión de orientación breve (aproximadamente 1 hora) con usuarios de la PYME piloto (administrador, personal de cocina y repartidores), explicando flujos principales y tareas de prueba.

3. Diseño de escenarios de prueba

Se definieron escenarios predefinidos para cubrir flujos críticos: (1) registro de pedido completo por cliente, (2) confirmación y preparación en cocina, (3) asignación y entrega por repartidor, (4) gestión de productos y categorías por administrador, (5) consulta al chatbot por WhatsApp. Cada

escenario utilizó datos simulados.

4. Ejecución de tareas guiadas

Los usuarios ejecutaron tareas bajo observación estructurada, registrando tiempos, errores y completitud de flujos. Las pruebas se realizaron en sesiones controladas durante un período acotado.

5. Recolección y análisis de datos

Se aplicó el cuestionario SUS al finalizar tareas guiadas y se consolidaron fichas de observación con métricas de desempeño. Se recopilaron logs para analizar disponibilidad, latencia y tasa de errores.

Tabla 12

Tareas evaluadas en pruebas de usabilidad

N.º	Tarea	Rol	Criterio de éxito
1	Crear un nuevo pedido	Cliente	Completar en < 3 min
2	Consultar estado de pedido	Cliente	Localizar en < 30 seg
3	Agregar producto al catálogo	Administrador	Completar sin errores
4	Asignar pedido a repartidor	Administrador	Completar en < 1 min
5	Marcar pedido como entregado	Repartidor	Completar sin ayuda
6	Realizar consulta al chatbot por WhatsApp	Cliente	Obtener respuesta pertinente

6. Respaldo de pruebas unitarias e integración

Se documentó evidencia técnica de ejecución de pruebas en backend con Jest y validaciones de integración de endpoints críticos. El resumen de ejecución se presenta en la Tabla 13; el detalle completo (salidas de consola, capturas y checklist de casos) se conservó en la bitácora técnica del proyecto.

Tabla 13*Evidencia resumida de pruebas unitarias y de integración*

Tipo	Módulo / alcance	Resultado	Evidencia registrada
Unitaria	AuthService, OrdersService, ProductsService	Aprobadas	Salida Jest + trazabilidad por identificador de caso
Integración	Flujo login + creación de pedido + cambio de estado	Aprobadas	Registro de requests/responses y verificación de estados en BD
Integración	Gestión de inventario por pedido y validación de stock	Aprobadas	Comparación antes/después en tablas de stock y logs de servicio
Integración	Acceso por roles (admin, cliente, repartidor)	Aprobadas	Pruebas de autorización con respuestas 200/401/403 según rol

Apéndice K

Guía de entrevista semiestructurada (diagnóstico Fase 1)

La guía de entrevista utilizada en el diagnóstico con responsables de las PYMES (capítulo Método, Fase 1; capítulo Resultados, estado de digitalización previo) se estructura por bloques temáticos: (1) datos generales del negocio y antigüedad; (2) gestión de pedidos (canales actuales, volumen, tiempos, errores); (3) comunicación con cocina y repartidores; (4) control de inventarios; (5) atención al cliente y herramientas digitales en uso. Las preguntas son abiertas para profundizar en procesos operativos, necesidades y expectativas. El instrumento completo en formato utilizable para replicación se incluye en el repositorio del proyecto o puede solicitarse a los autores.

Apéndice L

Arquitectura detallada de la plataforma

Este apéndice integra, en una sola vista textual, la arquitectura de la plataforma y las responsabilidades por capa para facilitar su revisión técnica sin depender únicamente de diagramas.

Tabla 14

Detalle de arquitectura por capas y componentes

Capa	Componente	Tecnología	Responsabilidad principal
Presentación	App móvil	React Native + Expo	Interfaz para cliente, cocina y repartidor; ejecución de flujos operativos.
Presentación	Panel web	Next.js + Tailwind CSS	Administración de catálogo, pedidos, repartidores y visualización de KPIs.
Negocio / API	Backend modular	NestJS + TypeScript	Reglas de negocio, seguridad JWT/RBAC, exposición de endpoints REST.
Datos	Persistencia transaccional	PostgreSQL (Supabase) + TypeORM	Gestión de entidades, relaciones e integridad de pedidos, usuarios e inventario.
Servicios externos	Mensajería y asistencia IA	WhatsApp Business API + servicio RAG	Atención automatizada y consultas frecuentes contextualizadas.
Infraestructura	Despliegue	VPS (Hetzner), Vercel, Docker	Operación en <i>staging</i> , publicación y continuidad técnica del prototipo.

La arquitectura se diseñó para mantener separación de responsabilidades, bajo acoplamiento entre capas y escalabilidad incremental, permitiendo evolucionar módulos sin afectar de forma directa el

resto de componentes.